

OpenACC*用户手册

Version 3.0

目录

1	简介.....	1
1.1	概述.....	1
1.2	范围.....	1
1.3	术语.....	1
1.4	执行模型.....	2
1.5	存储模型.....	3
1.6	兼容性.....	4
1.7	文档概述.....	4
1.8	引用文档.....	5
2	SWACC 编译系统.....	5
2.1	支持的基础语言标准.....	5
2.2	编译命令及选项.....	5
2.3	程序及编译运行示例.....	6
2.4	编译系统支持的常用功能.....	7
2.4.1	变量私有化分析功能.....	7
2.4.2	数组分布性分析功能.....	7
2.4.3	设备内存空间优化分析功能.....	8
2.4.4	Stencil 重用优化功能.....	8
3	OpenACC*编译指示.....	13
3.1	编译指示格式.....	14
3.2	条件编译.....	14
3.3	编译指示一览.....	14
3.4	加速计算区指示 parallel.....	17
3.4.1	if 子句.....	19
3.4.2	私有化子句 private/firstprivate.....	20
3.4.3	本地化子句 local.....	20
3.4.4	规约子句 reduction.....	20
3.4.5	缓存子句 cache.....	20
3.4.6	数据拷贝子句 copy/copyin/copyout.....	22
3.4.7	数据打包子句 pack/packin/packout.....	22
3.4.8	数组转置子句 swap/swapin/swapout.....	24
3.4.9	num_gangs 子句.....	26
3.4.10	num_workers 子句.....	27
3.4.11	异步子句 async.....	28
3.4.12	等待子句 wait.....	28
3.4.13	数据重用子句 present.....	28
3.4.14	分布规约子句 dis_reduction.....	28
3.4.15	暗示子句 annotate.....	28
3.4.16	不可缓存子句 uncache.....	28
3.5	循环映射指示 loop.....	28
3.5.1	gang 子句.....	30
3.5.2	worker 子句.....	31

3.5.3	向量化 vector 子句.....	32
3.5.4	循环合并子句 collapse.....	33
3.5.5	循环分块子句 tile.....	34
3.5.6	私有化子句 private.....	35
3.5.7	规约子句 reduction.....	35
3.5.8	分布规约子句 dis_reduction	35
3.5.9	暗示子句 annotate	35
3.6	加速数据区指示 data.....	35
3.6.1	if 子句	36
3.6.2	数据拷贝子句 copy/copyin/copyout	38
3.6.3	数据重用子句 present	38
3.6.4	本地化子句 local	38
3.6.5	异步子句 async.....	38
3.6.6	index 子句	38
3.6.7	数据分块子句 block.....	39
3.7	进入非结构数据区 enter data.....	41
3.7.1	async 子句.....	42
3.7.2	wait 子句.....	42
3.7.3	local 子句.....	42
3.7.4	copyin 子句.....	42
3.7.5	if 子句	42
3.7.6	swapahead 子句	42
3.7.7	swapalloc 子句.....	43
3.7.8	暗示子句 annotate	43
3.8	退出非结构数据区 exit data.....	43
3.8.1	async 子句.....	44
3.8.2	wait 子句.....	44
3.8.3	copyout 子句.....	44
3.8.4	if 子句	44
3.8.5	swapback 子句	44
3.8.6	暗示子句 annotate	44
3.9	数据更新指示 update.....	44
3.10	组合指示 parallel loop	45
3.11	原子操作指示 atomic	46
3.11.1	uncache 子句.....	47
3.12	等待指示 wait	47
3.13	函数指示 routine.....	48
3.13.1	memstack 子句.....	50
3.13.2	reuseldm 子句	51
3.13.3	uncache 子句.....	52
3.14	让出加速器控制权限 schedyield	52
3.15	探测指示 gapfinder.....	53
3.16	缓存刷新指示 flush	53
3.17	编译指示子句	53

3.17.1	私有化子句 private/firstprivate	54
3.17.2	本地化子句 local	56
3.17.3	规约子句 reduction.....	56
3.17.4	数据拷贝子句 copy/copyin/copyout	59
3.17.5	异步子句 async.....	63
3.17.6	等待子句 wait.....	65
3.17.7	数据重用子句 present	66
3.17.8	分布规约子句 dis_reduction	67
3.17.9	不可缓存子句 uncache.....	68
3.17.10	暗示子句 annotate	68
3.17.11	组合子句.....	80
4	运行时库接口.....	81
5	使用及编程指南.....	82
5.1	使用 OpenACC*进行应用移植和开发的步骤.....	82
5.2	程序移植示例	85
5.3	基于 OpenACC*的二次开发	88
5.4	关于加速线程数据 Cache 的使用方式	89

1 简介

1.1 概述

欢迎使用 OpenACC*语言及 SWACC 编译系统。本文档主要介绍 SWACC 编译系统的使用以及其所支持的 OpenACC*语言的用法和说明。OpenACC*语言是在 OpenACC2.0 文本的基础上，针对申威众核处理器结构特点进行适当的精简和扩充而来的。OpenACC 是由 OpenACC 组织(www.openacc-standard.org)于 2011 年推出的众核加速编程语言，2013 年发布 2.0 文本。OpenACC 是以编译指示的方式提供众核编程所需的语言功能，其主要目的是降低众核编程的难度。

1.2 范围

本文档所描述的编译指示、用户接口以及使用方式只针对加速编程，即将编译指示的指定的程序代码段加载到加速设备上执行，程序的其他部分仍然在主处理器上执行。本文档只介绍对加载到加速设备上的代码的详细要求。对在主处理器上编程的特征和限制，本文档不作说明。

本文档不讲述使用编译器或其他工具自动检测并将代码段加载到加速设备运行的内容，也不介绍如何将加速代码分配到多个加速设备上。

1.3 术语

加速设备 (Accelerator device): CPU 的特殊协处理器，简称为 Device。CPU 可以将数据与计算核心下载到该特殊协处理器上执行，如 GPU、MIC、申威众核处理器的运算核心阵列（简称从核阵列）等都可以称为加速设备。

主处理器 (Host): 主处理器通常指通用处理器核心，如通用 CPU、申威众核处理器的运算控制核心（简称主核）。在本文档中特指连接有加速设备的主处理器。

主存 (Host memory): 主处理器的内存空间。

设备内存 (Device memory): 位于加速设备内的内存。加速设备访问设备内存通常比访问主存快，如 GPU 中的 shared memory、申威众核处理器中的运算核心局存(LDM, Local Data Memory)。

主线程 (Host thread): 在主处理器上运行的程序实体，简称为主进程、或主线程、或进程。

加速线程 (Accelerator thread): 运行在加速设备处理器核心上的程序实体，简称为线程。

主程序代码 (Host code): 运行在主处理器上的程序代码，简称主程序或主程序代码。在本文档中所指的程序代码，除指定要在加速设备上执行的加速程序代码外，均为主程序代码。主程序代码控制加速程序代码的加载运行。

加速程序代码 (Accelerator code): 运行在加速设备上的程序代码，也称为加速程序/代码。

加速任务 (Kernel): 在加速设备上运行的代码和数据称为加速任务。在不同的加速设

备上，kernel 可以是简单的循环或复杂的子例程。

结构化代码块 (Structured block): 结构化代码块是有单一入口和单一出口的复合语句；对于 C 程序来说，即为用一对大括号包含的代码段。

加速计算区 (Accelerator compute region): 由加速计算区编译指示定义的一个代码区域。加速计算区是一个在加速设备上执行的结构化代码块。加速计算区通常要求在入口处向设备内存申请空间，并将数据从主存拷贝到设备内存；在出口处将数据从设备内存拷贝回主存并释放设备内存空间。加速计算区不可以包含其它加速计算区，但可以包含加速循环。在 OpenACC*语言中，parallel 指示标注的代码区域即为加速计算区，也是需要在加速设备上运行的加速程序代码(Accelerator code)，经过编译器的处理之后，加速计算区的程序代码将被封装成加速任务(Kernel)，最终加载到加速设备上运行。

加速数据区 (Accelerator data region): 由加速数据区编译指示定义的一个区域。加速数据区通常要求在入口处向设备内存申请空间，并将数据从主存拷贝到设备内存；在出口处将数据从设备内存拷贝回主存并释放设备内存空间。加速数据区可以包含其它加速数据区和加速计算区。

加速循环 (Accelerator loop): 由循环映射编译指示定义的一个循环体，位于加速计算区内。加速循环通常会以任务分担的方式被分配到多个加速线程上执行。

私有变量 (Private variable): 指存放在线程私有主存空间内，仅本加速线程可以访问的变量。变量在私有化后，每个线程都会有与主线程变量相对应的副本，在并行运行时不会发生写冲突的情况。

本地变量 (Local variable): 指存放在加速设备的设备内存中，仅本加速线程可以访问的变量，属于私有变量的一种特例。

1.4 执行模型

OpenACC*程序的执行模型是在 host（主处理器）的指导下，host 和 device（加速设备）协作的加速执行模型，如图 1-1 所示。程序首先在 host 上启动运行，以一个主线程串行执行，或者通过使用 OpenMP 或 MPI 等编程接口以多个主线程并行执行，计算密集区域则在主线程的控制下作为 kernel（加速任务）被加载到加速设备上执行。kernel 的执行过程包括：在设备内存上分配所需数据空间；加载 kernel 代码（包含 kernel 参数）至 device；kernel 将所需数据从主存传输至设备内存；等待数据传输完成；device 进行计算并将结果传回主存；释放设备上的数据空间等。大多数情况下，host 可以加载一系列 kernels，并在加速设备上逐个执行。

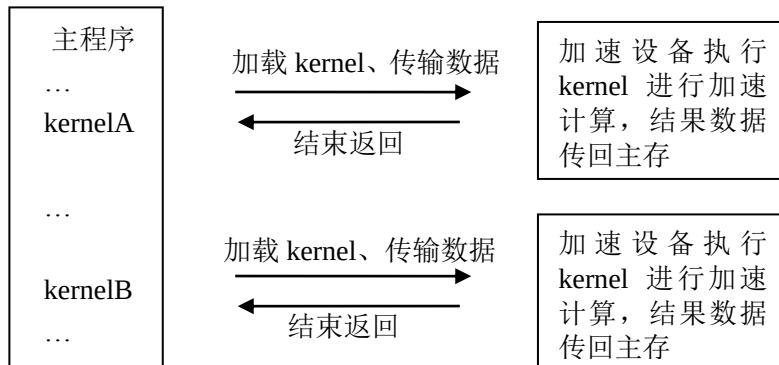


图 1-1 执行模型示意图

OpenACC*支持两级并行机制：gang、worker。gang 是粗粒度并行，在加速设备上可以启动一定数量的 gang。worker 是细粒度并行，每个 gang 内包含有一定数量的 worker。gang、

worker 两级并行是包含关系，对于一层循环并行可以不用指定，对于嵌套循环并行则要遵循两者的嵌套关系，即 **gang** 级并行的循环内只允许出现 **worker** 级并行的循环。

在申威众核处理器中，一个运算控制核心（简称主核）仅控制一个运算核心阵列(加速设备，简称从核阵列)的运行，每个运算核心阵列内有 64 个运算核心（简称从核），每个运算核心可以运行一个加速线程。默认情况下，64 个加速线程被组织成 64 个 **gang**、每个 **gang** 内一个 **worker** 并行的逻辑视图。**gang** 的数量与 **worker** 大小的乘积即为实际运行的加速线程的数量，但是不可以超出 64。通常情况下，尽量用满 64 个加速线程可以获得较好的运行性能。

1.5 存储模型

仅在 Host 上运行的程序和 Host+Device 上运行的程序，二者最大的区别在于存储层次和存储模型的不同。当前大多数 Host+Device 平台，比如 CPU+GPU 平台，支持的存储模型为主存和设备内存分离的模型，**device** 不能直接访问主存，加速计算所需的数据必需通过 Host 进行传输。申威众核处理器采用了片上融合异构众核体系结构，**device** 可直接访问主存空间，并在 **device** 内提供加速线程私有的高速缓冲(LDM, Local Data Memory)，加速计算需要存放到 LDM 的数据由 **device** 控制传输。本系统支持的存储模型如图 1-2 所示。

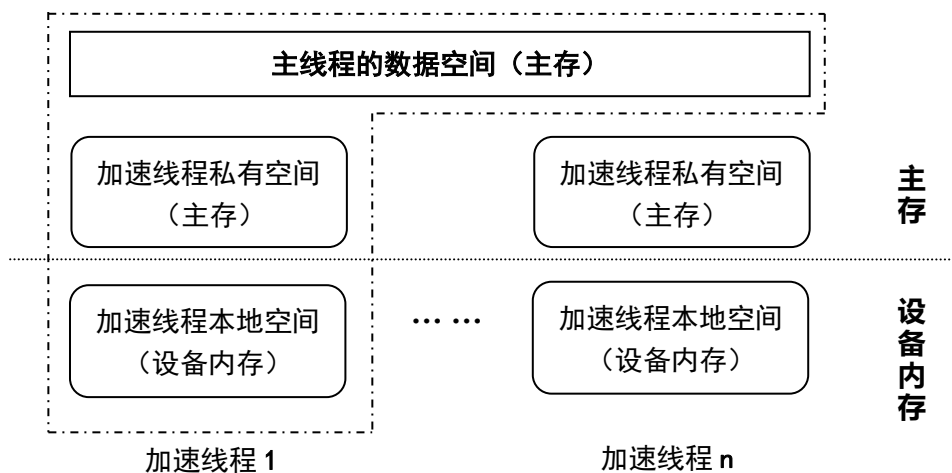


图 1-2 存储模型示意图

图 1-2 中虚线框内为一个加速线程可见的存储空间，包括三个部分：

- 1) **主线程数据空间**：位于主存，主线程的内存空间对其创建的加速线程直接可见，加速线程可以直接访问相应的数据；对于主线程创建的多个加速线程而言，这部分空间是共享的；程序中在加速区外定义的变量，均位于该空间内。
- 2) **加速线程私有空间**：位于主存，每个加速线程有独立的私有空间，程序中使用 `private`、`firstprivate` 子句修饰的变量将存放于该空间内。
- 3) **加速线程本地空间**：位于设备内存，每个加速线程有独立的本地空间(LDM)，本地空间的访问性能是三种空间中最高。程序中使用 `local`、`copy` 等数据子句修饰的变量将由编译系统控制全部或局部存放于 LDM 内。主存与本地空间的数据交互由加速线程控制。

使用 OpenACC*编写程序，数据的管理和传输是隐式的，由编译器根据编译指示的信息，自动生成空间管理、数据传输等控制代码，然而充分了解系统的存储模型和特点，对编写出高性能的众核应用十分重要，需要了解的一些加速编程的注意事项包括：

- 1) 一个高效的加速代码需要有一定的计算密度，需要多高的计算密度取决于主存和设备内存之间的访存带宽；
- 2) 设备空间的容量有限，编写程序时需要充分考虑设备空间的容量和加速代码的内存需求。

1.6 兼容性

由于在申威众核处理器上程序的执行方式和存储模型与 GPU、MIC 等平台存在一定的差异，主要体现在存储层次、数据传输控制方式上（参看 1.4 和 1.5），而 OpenACC2.0 文本中约定的存储模型主要针对 GPU 这类加速器的结构特点设定。因此本系统在支持 OpenACC2.0 方面有以下一些特点：

- 1) SWACC 编译系统支持 OpenACC2.0 文本全集，包括其中定义的编译指示、运行时库接口和环境变量，即按照 OpenACC2.0 文本规范，编写的基于 GPU 或其他平台的程序，可以直接使用 SWACC 编译系统进行编译并运行，保证其主要语言功能的含义和程序的正确性。
- 2) 存储模型的映射：OpenACC2.0 文本中主要针对存储模型是主存和设备内存分离的模型，加速设备不能访问主存；而申威众核处理器支持的片上融合异构众核体系结构使得加速设备可以直接访问主存，按照 OpenACC2.0 文本的说明，在加速设备和主进程共享主存的平台上，OpenACC2.0 文本中在设备内存上申请空间和数据传输等操作在实现时不需要实际执行。在申威众核处理器中设备可用的内存主要包括两部分：主存和设备内存，在实际实现时将这两部分作为整体对应到 OpenACC2.0 文本中所定义的“设备内存”，由编译系统决定具体变量的存储位置。
- 3) 执行模型的映射：OpenACC2.0 文本定义的执行模型与申威众核处理器的执行模型是基本一致的，因此其中的加速计算区等相关功能可直接对应到申威众核处理器的执行部件。

在支持 OpenACC2.0 文本基础上，为了更好的利用 OpenACC2.0 未能在语言功能中体现、但对提升程序性能十分关键的申威众核处理器的特征和用法，本系统对 OpenACC2.0 文本的功能进行了适当且必要的扩充、对一些语言功能的使用方式进行了扩展，本文档后续的内容不按照 OpenACC2.0 文本的格式进行说明，将主要介绍在申威众核处理器上如何开发出高效的 OpenACC*加速应用所需要了解和掌握的语言功能和注意事项。

1.7 文档概述

阅读本手册前，希望读者对 C、Fortran、OpenMP 语言比较熟练，对 UNIX/LINUX 操作系统进程和并行机制有一定的了解。本手册重点叙述 SWACC 编译系统和 OpenACC*语言的使用方法，本文档后续的主要内容包括：

- 2 SWACC 编译系统
- 3 OpenACC*编译指示
- 4 运行时库接口
- 5 使用及编程指南
- 6 版权说明

1.8 引用文档

- 1) American National Standard Programming Language C, ANSI X3.15-1989(ANSI C).
- 2) ISO/IEC9889:1999, Information Technology – Programming Languages – C(C99).
- 3) ISO/IEC14882:1998, Information Technology – Programming Languages – C++.
- 4) ISO/IEC 1539:1980, Information Technology - Programming Languages – Fortran (Fortran77)
- 5) ISO/IEC 1539:1991, Information Technology - Programming Languages – Fortran (Fortran90).
- 6) ISO/IEC 1539-1:1997, Information Technology - Programming Languages – Fortran (Fortran95).
- 7) The OpenACC Application Programming Interface, version 2.0, 2013.06.

2 SWACC 编译系统

SWACC 编译系统是支持 OpenACC*语言的基础，使用 OpenACC*语言编写的程序需要使用 SWACC 编译系统进行编译。同时 SWACC 编译系统支持使用 OpenACC*与 MPI 等编程接口编写的混合程序的编译。

2.1 支持的基础语言标准

SWACC 编译系统支持的基础语言标准包括：

- 1) C 语言；
- 2) Fortran 语言；
- 3) C++语言(运算核心不支持 C++，即加速代码不支持 C++)。

2.2 编译命令及选项

SWACC 编译系统提供三种编译命令：**swacc**、**swaCC**、**swafort**，分别对应 C、C++、Fortran 程序的编译。以编译 C 程序为例，其编译命令的格式为：

swacc [选项] 文件名

其中，选项和文件名为输入参数，中括号表示为可选的参数，比如 **swacc hello.c**。

SWACC 编译系统支持的编译选项包括自有的编译选项和通用编译选项。通用编译选项为 cc、gcc 和 icc 中提供的常规编译选项，包括：**-c -g -Olevel -o -Idir -Ldir -Dmacro[=defn]** 等。SWACC 编译系统主要的自有编译选项如表 2-1 所示。

表 2-1 SWACC 编译选项

编译选项	说明
--(h help)	显示帮助
-SCFlags	指定的选项将被传递到编译 device 程序的串行编译器，若同时传递多个选项需使用逗号进行分割，中间不能有空格，例如： swafort -SCFlags -extend_source,-O3 hello.c
-HCFlags	指定的选项将被传递到 host 程序的基础编译器，若同时传递多个选项需使用逗号进行分割，中间不能有空格
-LFlags	指定的选项将被传递到链接器，若同时传递多个选项需使用逗

	号进行分割，中间不能有空格
-v -version	显示编译器和运行时库的版本号
-keep	保留编译的中间文件
-dumpcommand dumpfile	将对中间文件的编译命令输出到 dumpfile 中
-profiling	swacc 性能分析功能选项，需配合 profiling 暗示子句使用，用于进行性能检测代码插装，统计和分析加速段性能
-profiling_mpirank profrank	针对 MPI+OpenACC 程序，指定对 mpi 全局逻辑进程号等于 profrank 的进程进行性能统计和分析，默认对 mpirank=1 的进程进行性能统计分析。需要配合 -profiling 选项和 profiling 暗示子句使用
-chkerr	运行时错误检查选项，检查常见的、由于用法不当导致的程序运行异常或结果错，用法为使用该选项编译所有加速区涉及的文件，然后运行即可

[注：通用编译选项可使用命令 `man gcc` 等查看相关功能]

2.3 程序及编译运行示例

本节通过一个简单的示例程序(文件名为 `ex_2_3.c`)介绍如何使用 SWACC 编译系统编译及运行 OpenACC*程序。

```

1 /*
2  * FILE:ex_2_3.c
3  */
4
5 #include <stdio.h>
6 #define N 1024
7
8 int A[N];
9
10 main()
11 {
12     int i;
13     long result;
14
15     #pragma acc parallel loop copyout(A)
16     for(i = 0; i < N; i++)
17         A[i] = i;
18
19     result=0;
20     for(i = 0; i < N; i++)
21         result+=A[i];
22
23     if(result == (N-1)*N/2)
24         printf("test OK!\n");
25     else
26         printf("test FAILED!\n");
27
28     return 0;
29 }
```

图 2-1 简单的 OpenACC*示例

图 2-1 中，第 15 行为 OpenACC*的组合加速编译指示 `parallel loop`，用于指定紧随其后

的加速计算区，并且加速计算区内仅有一个加速循环，第 16-17 行为对应的加速循环体。其中 `copyout(A)` 子句表明 A 数组将在设备内存中申请必要的空间，并在计算结束后将设备内存中 A 数组的数值拷贝回主存中对应的地址空间。

将图 2-1 中的程序保存为 `ex_2_3.c`，使用下面编译命令进行编译：

```
swacc ex_2_3.c
```

SWACC 编译系统会将加速计算区的代码封装为加速任务，对于加速循环中的循环和相应的数据会以最优的方式映射到加速设备上运行，默认生成可执行文件 `a.out`。

提交运行命令如下：

```
bsub -I -q q_test -n 1 -np 64 -priv_size 4 ./a.out
```

其中 `bsub` 为系统提交作业的命令，`-q q_test` 为指定提交作业所需的队列名，`-n 1` 指定启动一个主进程运行，`-np 64` 指定需要 64 个加速线程进行加速（作业提交运行的命令仅供示例说明和参考，具体使用方式参考相关手册）。

使用 OpenACC* 进行应用移植开发的步骤和注意事项，请参考第 5 章的内容，对于初次接触 OpenACC* 语言的用户建议优先阅读。

2.4 编译系统支持的常用功能

2.4.1 变量私有化分析功能

在本文档所述的 OpenACC* 加速编程模型中，主存空间默认是主进程和加速线程共享的，因此准确确定私有化变量直接关系到程序运行的正确性。用户可以在编译时添加编译选项 `-priv`，SWACC 编译系统将对标注 `acc parallel loop` 的核心代码段中需要私有化的变量、只读标量和只读数组进行自动分析。若存在无法分析的变量，则需要用户分析并确定相关属性以确保程序执行结果的正确性。图 2-2 是使用编译选项 `-priv` 对某程序中标注 `acc parallel loop` 的核心段进行私有化分析的结果。

```
编译指示位置信息：/ldata/zlb/for_swacc_tune/gkufs/algorithm.f：94
需要私有化的变量列表：jcbijk,FDVX,FDV0,FDV1,FDVNi,jk,cfijk,cf2dt,i,j,k,cpr,RONijk,Uijk,Vijk,Wijk,Tijk,Pijk,vijk2,cpt,crt,viu,vjv,vkw,FDVM,FVQ;建议您将其加入到private或者local子句
书写格式：local(jcbijk,FDVX,FDV0,FDV1,FDVNi,jk,cfijk,cf2dt,i,j,k,cpr,RONijk,Uijk,Vijk,Wijk,Tijk,Pijk,vijk2,cpt,crt,viu,vjv,vkw,FDVM,FVQ)
只读标量列表：im,jm,km,DThalf,DTsource,iv,jv,kv,PI,RCON,vi_sj,vj_sj,vk_sj;建议您将其加入到copyin子句和annotate的readonly子句
书写格式：copyin(im,jm,km,DThalf,DTsource,iv,jv,kv,PI,RCON,vi_sj,vj_sj,vk_sj) annotate(readonly=(im,jm,km,DThalf,DTsource,iv,jv,kv,PI,RCON,vi_sj,vj_sj,vk_sj))
只读数组列表：CF,jcb,S
分析不清楚的变量列表：FDV
```

图 2-2 编译选项 `-priv` 自动分析示例

如图 2-2 所示，SWACC 编译系统列出了相关子句的书写格式及调优建议，用户可以在确认编译器分析结果正确后，将其添加到 `acc parallel loop` 的编译指示中。

2.4.2 数组分布性分析功能

用户在编译时可以添加编译选项 `-arrayAnalyse`，SWACC 编译系统将自动对标注 `acc parallel loop` 的核心代码段中可以拷贝到加速设备的数组变量及需要私有化的数组变量进行

分析，图 2-3 是对标注 acc parallel loop 的核心段进行数组分布性分析的结果。

```
-----
编译指示位置信息：/ldata/zlb/for_swacc_tune/gkufs/algorithm.f : 95
可以copy的变量列表：FDV,CF,jcb,S
-----
```

图 2-3 编译选项-arrayAnalyse 自动分析示例

用户可以结合私有化分析结果，对可 copy 的变量进行进一步的细分，如果数组位于只读数组列表，那么将其加入到 copyin 子句，否则就加入到 copy 子句。

2.4.3 设备内存空间优化分析功能

用户在编译时如果添加编译选项-ldmAnalyse 进行编译，那么程序在运行时将反馈给用户当前设备内存的详细使用情况并自动分析出一种较优的循环及数据分块值（关于循环分块，可参考 3.5.5 循环分块子句 tile 的说明，图 2-4 是对标注 acc parallel loop 的核心段进行设备内存优化分析的结果示例。

```
=====
LDM 空间优化分析结果：
acc region 信息：
  文件位置信息：/home/export/base/pfsacc/zlb/gkufs/algorithm.f : 94
  编译器参数使用空间大小：24 字节
  region上的数据拷贝变量列表：

      im      jm      km      DThalf      DTsource      iv      jv      kv
      PI      RCON     vi_sj     vj_sj     vk_sj     jcbijk     FDVX     FDV0
      FDV1     FDVNijk    cfijk     cf2dt     i         j         k         cpr
      RONijk    Uijk     Vijk     Wijk     Tijk     Pijk     vijk2     cpt
      crt      viu      vjv      vkw      FDVM      FVQ

  region上数据拷贝变量所使用的LDM空间大小：152 字节
acc do 信息：
  文件位置信息：/home/export/base/pfsacc/zlb/gkufs/algorithm.f : 95
  block指示信息：block(k:1,j:1)
  acc do上分布数组列表：

      FDV(64,1,1,12)      CF(64,1,1)      S(19,64,1,1)      jcb(64,1,1)
  acc do上分布数组所使用的LDM空间大小：8448 字节
总计：已使用的LDM空间大小为：8624 字节
我们建议使用的block指示信息如下：
  文件位置信息：/home/export/base/pfsacc/zlb/gkufs/algorithm.f : 95
  block指示信息：block(k:1,j:2)
=====
```

图 2-4 ldm 空间优化分析结果示例

2.4.4 Stencil 重用优化功能

用户程序里面如果含有 stencil 计算，可以在编译时添加-stencil_opt 选项，编译器会针对 stencil 计算做数据重用优化，从而减少 dma 数据量，提升程序性能。（Stencil 计算示例：BB[i][j] = AA[i][j] + AA[i-1][j] + AA[i+1][j] + AA[i][j+1] + AA[i][j-1]）目前编译器可以识别 i-a/i+b 且 a=b 类型的 stencil 计算，下标带系数的情况不支持。

Stencil 重用优化的原理及效果是，当存在类似[i-1]、[i]、[i+1]这种访问时，当前 i 的部分数据在执行 i+1（记为 i'）时，仍然能使用，[i]、[i+1]变成了[i'-1]、[i']，stencil 重用优化就是寻找或者创造这种可以重复使用数据的机会，从而达到减少数据传输的目的。

此小节建议在阅读完第 3 章内容之后再阅读。

Stencil 重用优化对象是通过 parallel 上 copy 或者 data copy 拷贝的数组。

1、如果需要优化的数据通过 **parallel loop** 上的 **copy** 拷入局存，只需要编译时使用 **-stencil_opt** 选项。示例如下。

```

1 //FILE:ex_2_4_4-01.c
2 #include <stdio.h>
3 #define NX 512
4 #define NY 512
5 int AA[NX][NY];
6 int BB[NX][NY];
7 void main()
8 {
9
10  int i, j;
11
12  for(i=0;i<NX;i++)
13      for(j=0;j<NY;j++)
14          AA[i][j]=i+j;
15
16  #pragma acc parallel loop copyin(AA) copyout(BB)
17  for(i=1;i<NX-1;i++)
18  {
19      for(j=1;j<NY-1;j++)
20      {
21          BB[i][j] = AA[i][j] + AA[i-1][j] + AA[i+1][j] \
22                      + AA[i][j+1] + AA[i][j-1];
23      }
24  }
25
26
27  for(i=1;i<NX-1;i++)
28  {
29      for(j=1;j<NY-1;j++)
30      {
31          if(BB[i][j] != 5*(i+j))
32          {
33              printf("error!\n");
34              exit(-1);
35          }
36      }
37  }
38
39  printf("TEST OK!\n");
40  FILE* fp = fopen("TEST_RIGHT", "w");
41  fclose(fp);
42
43 }

```

图 2-5 stencil 优化示例程序一

以上程序中可以做 stencil 重用优化的是 AA 数组，由于该数组是在 parallel loop 上 copyin 进来的，因此直接加 **-stencil_opt** 编译优化即可生效。

2、如果需要优化的数据是通过 **data copy** 拷入局存的，那么需要在 data copy 上指定需要拷入的数据相应维度上需要拷贝的范围，且 lower、upper、max_length 三个参数都需要有，格式见 3.17.4。例如：

```

1 //FILE:ex_2_4_4-02.c

```

```

2 #include <stdio.h>
3 #define NX 512
4 #define NY 32
5 #define NZ 16
6 #define NN 32
7 int AA[NX][NY][NZ][NN];
8 int BB[NX][NY][NZ][NN];
9 int BB1[NX][NY][NZ][NN];
10
11
12
13 void main()
14 {
15
16     int i, j, k, l, m;
17
18     for(i=0; i<NX; i++)
19         for(j=0; j<NY; j++)
20             for(k=0; k<NZ; k++)
21                 for(l=0; l<NN; l++)
22                     AA[i][j][k][l]=i+j+k+l;
23
24
25 //HOST :
26     for(i=1; i<NX-1; i++)
27     {
28         for(j=1; j<NY-1; j++)
29         {
30             for(k=1; k<NZ-1; k++)
31             {
32                 for(l=1; l<NN-1; l++){
33                     BB1[i][j][k][l] = AA[i][j][k][l] + \
34                                     AA[i+1][j][k][l] + \
35                                     AA[i-1][j][k][l] + \
36                                     AA[i][j+1][k][l] + \
37                                     AA[i][j-1][k][l] + \
38                                     AA[i][j][k+1][l] + \
39                                     AA[i][j][k-1][l];
40                 }
41             }
42         }
43     }
44
45
46 //ACC :
47     #pragma acc parallel loop copyout(BB)
48     for(i=1; i<NX-1; i++)
49     {
50         for(j=1; j<NY-1; j++)
51         {
52             #pragma acc data copyin(AA[i-1:i+1:3][j-1:j+1:3][*][*])
53             {
54                 for(k=1; k<NZ-1; k++)
55                     for(l=1; l<NN-1; l++){
56                         BB[i][j][k][l] = AA[i][j][k][l] + \
57                                         AA[i+1][j][k][l] + \
58                                         AA[i-1][j][k][l] + \

```

```

59                                     AA[i][j+1][k][l] + \
60                                     AA[i][j-1][k][l] + \
61                                     AA[i][j][k+1][l] + \
62                                     AA[i][j][k-1][l];
63                                 }
64                            }
65                       }
66                   }
67       }
68
69       for(i=1;i<NX-1;i++)
70       {
71           for(j=1;j<NY-1;j++)
72           {
73               for(k=1;k<NZ-1;k++)
74               {
75                   for(l=1;l<NN-1;l++)
76                   if(BB1[i][j][k][l] != BB[i][j][k][l])
77                   {
78                       printf("TEST FAIL!!\n");
79                       exit(-1);
80                   }
81               }
82           }
83       }
84   }
85
86   printf("TEST OK!\n");
87   FILE* fp = fopen("TEST_RIGHT", "w");
88   fclose(fp);
89
90 }

```

图 2-6 stencil 优化示例程序二

以上程序可以做 stencil 重用优化的数组是 AA，该数组是通过 data copy 语句拷进局存的，因此如果想要优化生效，需要在 data copy 中指定待优化维度的下标范围，即 copyin(AA[i-1:i+1:3][j-1:j+1:3][*][*])，且 j-1/j+1/3 三个参数都需要有，优化将对被指定下标范围的最内层生效，即 j 层。

3、支持 data copy 上带有数据分块的 stencil 优化，需要注意的是该种情况下，数据分块后的分块外提功能不能被关闭(分块外提功能参见 3.6.7)，否则该 stencil 优化将没有意义。例如：

```

1 /*
2 * FILE:ex_2_4_4-stream11.c
3 */
4 #include <stdio.h>
5 #define NX 512
6 #define NY 128
7 #define NZ 64
8 #define NN 32
9
10 int AA[NX][NY][NZ][NN];
11 int BB[NX][NY][NZ][NN];
12 int BB1[NX][NY][NZ][NN];
13

```

```

14 int CC[NN];
15 int DD[NN];
16
17 int EE[NX][NY][NZ];
18
19 void main()
20 {
21
22     int i, j, k, l, m;
23
24
25
26     for(i=0; i<NX; i++)
27         for(j=0; j<NY; j++)
28             for(k=0; k<NZ; k++)
29                 for(l=0; l<NN; l++)
30                     AA[i][j][k][l]=i+j+k+l;
31
32     for(l=0; l<NN; l++)
33     {
34         if(l%2 == 0)
35             CC[l] = 1;
36         else
37             CC[l] = -1;
38     }
39
40     for(l=0; l<NN; l++)
41     {
42         DD[l] = (l*l)% (NN-1);
43     }
44
45     for(i=0; i<NX; i++)
46         for(j=0; j<NY; j++)
47             for(k=0; k<NZ; k++)
48                 EE[i][j][k]=i+j+k;
49
50 //HOST time:
51     for(i=1; i<NX-1; i++)
52     {
53         for(j=1; j<NY-1; j++)
54         {
55             for(k=1; k<NZ-1; k++)
56                 for(l=1; l<NN-1; l++){
57                     m = DD[l];
58                     BB1[i][j][k][l] = AA[i+CC[m]][j+CC[m]][k][l] ;
59                 }
60         }
61     }
62
63
64 //ACC time:
65 #pragma acc parallel copyin(DD,CC) annotate(entire(DD,CC))
66 #pragma acc loop
67     for(i=1; i<NX-1; i++)
68     {
69         #pragma acc loop tile(5)
70         for(j=1; j<NY-1; j++)

```

```

71      {
72          #pragma acc data copyin(AA[i-1:i+1:3][j-1:j+1:3][*][*])
copyout(BB[i][j][*][*]) block(k:10)
73          {
74              for(k=1;k<NZ-1;k++)
75                  for(l=1;l<NN-1;l++){
76                      m = DD[l];
77                      BB[i][j][k][l] =
AA[i+CC[m]][j+CC[m]][k][l];
78                  }
79              }
80          }
81      }
82  }
83
84  for(i=1;i<NX-1;i++)
85  {
86      for(j=1;j<NY-1;j++)
87      {
88          for(k=1;k<NZ-1;k++)
89          {
90              for(l=1;l<NN-1;l++)
91              {
92                  if(BB1[i][j][k][l] != BB[i][j][k][l])
93                  {
94                      printf("TEST FAIL!!\n");
95                      exit(-1);
96                  }
97              }
98          }
99      }
100
101      printf("TEST OK!\n");
102      FILE* fp = fopen("TEST_RIGHT", "w");
103      fclose(fp);
104
105  }

```

图 2-7 stencil 优化示例程序三

上例中，stencil 重用优化将会在存在 stencil 计算的最内层 j 进行，AA、BB 数组的 k 维将会被分块，从而导致 k 层循环被分块，该循环分块的外层循环将会被提至 parallel 作用范围的最外层（即 i 层循环的外层），内层循环的块大小是 10，在该分块的外层循环的一次迭代内，j 维将每次对块大小为 10 的 k 维数据进行重用。如果外提功能被关闭，那么每次 dma 的 k 维参数都会发生变化，j 维的 stencil 重用将无法实现。

3 OpenACC*编译指示

本章介绍 OpenACC*编译指示的语法和使用方式，在 C 和 C++ 中，用 #pragma 机制来描述 OpenACC*编译指示。在 Fortran 中，用带有独特前导符的注释来描述 OpenACC*编译指示。对于不支持或者关闭了 OpenACC*的编译器而言，OpenACC*编译指示将被忽略。

3.1 编译指示格式

在 C/C++ 中，使用 **#pragma** 机制描述 OpenACC* 指示，其语法格式为：

#pragma acc 编译指示名 [子句[[,] 子句]...] 换行

每个编译指示均以 **#pragma acc** 开头。编译指示的其它部分都遵守 C/C++ 中 **pragma** 的使用规范，# 的前后都可以使用空白字符；编译指示的多个子句之间使用空白字符或逗号分隔。在 C/C++ 程序中编译指示区分大小写。OpenACC* 编译指示作用于紧接着的语句、结构块和循环。C/C++ 语言中使用 “\” 作为编译指示的续行符。

在 Fortran 语言自由格式的程序中，以 **!\$acc** 前导符作为 OpenACC* 编译指示的起始标记，其语法格式为：

!\$acc 编译指示名 [子语[[,] 子语]...] 换行

第一个注释字符 **!** 可以出现在一行的任意列，但它前面只能是空白字符(空格和跳格)。前导符 **!\$acc** 必须以一个整体出现，中间不能有空白字符。每行的长度、空白字符、续行符规则同样适用于编译指示行。编译指示起始行的前导符 (**!\$acc**) 后必须有一个空白字符。待续行中导语部分的最后一个非空白字符必须是续行符 (**&**)，续行符后面仍然可以写注释；接续行中必需以前导符开始(前面允许有空白字符)，前导符后面的第一个非空白字符可以是续行符。

在 Fortran 固定格式的程序中，OpenACC* 编译指示可以采用下列形式中的一种：

!\$acc 编译指示名 [子语[[,] 子语]...] 换行

c\$acc 编译指示名 [子语[[,] 子语]...] 换行

***\$acc 编译指示名 [子语[[,] 子语]...] 换行**

前导符 (**!\$acc**, **c\$acc**, ***\$acc**) 必需写在 1-5 列，固定格式的每行长度、空白字符、续行、列的规则同样适用于编译指示行。编译指示起始行的第 6 列必须是空格或者 0，接续行第 6 列不能是空格或 0。

在 Fortran 程序中编译指示不区分大小写。后续章节中 Fortran 相关的用法及示例均以自由格式进行说明。

每个编译指示中只能出现一个编译指示名，多个子句出现的顺序无关紧要，除非特别限定，同一个子句可以重复出现多次。

3.2 条件编译

当使用 SWACC 编译系统编译 OpenACC* 程序时，**_OPENACC** 宏被定义，其值由编译器确定。

3.3 编译指示一览

OpenACC* 的编译指示及子句如图 3-1 所示。

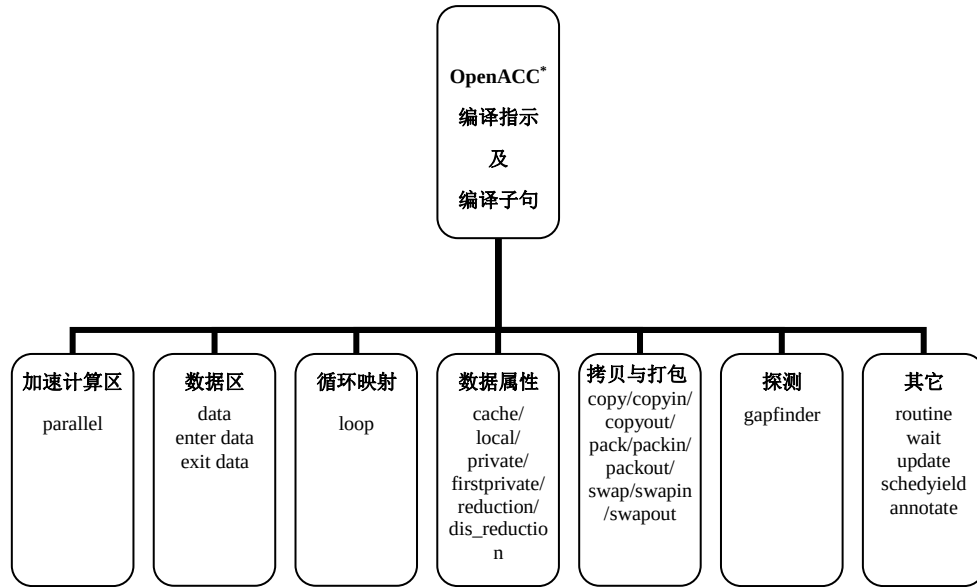


图 3-1 OpenACC*编译指示及编译子句概览图

图 3-1 是 OpenACC*加速编程语言的主要编译指示及编译子句概览图，OpenACC*编译指示和子句的功能如表 3-1 和表 3-2。

表 3-1 编译指示

编译指示	说明	子句
parallel	用在一段代码之前，表示这段代码将作为加速任务加载到加速设备上执行。	if private firstprivate reduction cache local copyin/copy/copyout packin/pack/packout swapin/swap/swapout wait async num_gangs num_workers present annotate
data	在 parallel 作用范围内使用有效，描述控制数据传输。	if copyin/copy/copyout present async
loop	用于循环前面，标记加速循环。	private reduction tile gang

		worker collapse annotate
parallel loop	parallel 和 loop 指示的组合，用在一个循环之前，表示循环的代码将被多个加速线程并行执行。	parallel 和 loop 指示子句的组合
atomic	用于指定一块内存区域被自动更新。	无
routine	用于说明将在加速计算区中被调用的函数。	无
enter data	进入非结构数据区	async wait local copyin if swapahead swapalloc annotate
exit data	进入非结构数据区	async wait copyout if swapback annotate
update	用于更新或访问非结构数据区中的数据。	async wait host device if annotate
wait	等待在此语句之前运行的异步任务结束。	async
schedyield	退出对加速设备的控制。	无
gapfinder	全局操作探测函数，目前仅配合 dis_reduction 使用，检查是否有需要处理的请求。	无

表 3-2 编译指示子句

编译指示子句	说明
private	将一个或若干个变量指定为加速线程私有变量
firstprivate	将一个或若干个变量指定为加速线程私有变量，并且变量赋初值，初值为主进程中的值
reduction	指定一个或多个变量是私有的，并且在并行处理结束后这些变量要执行指定的运算
dis_reduction	分布规约操作；针对长度较大、需要进行更新（读修改写、比如+=或-=）操作的数组
if	如果 parallel 指示存在 if 子句，则加速区中的代码当且仅当 <逻辑表达式>值为 TRUE 时并行执行，其它情况下，加速区被串行执行；

	如果 data 指示存在 if 子句，则当且仅当<逻辑表达式>值为 TRUE 时发生数据的拷贝，其他情况下，不发生数据拷贝
cache	暗示对应的变量非常重要，需尽可能放到最快的内存中，如局存、软 cache ，使用最有效的存取方式，具体由编译器优化时决定
local	Local 变量列表中的变量、数组等，必须在设备内存中申请空间，改变主存中同名变量的值不会影响到加速设备，加速设备上对这些变量的值做的改变也不影响 host
copy/copyin/copyout	可指定把标量、数组、数组的全部或部分在主存和局存之间拷贝，该子句可跟在 parallel 、 data 多个编译指示后，语义相同，但限制可能有所不同
pack/packin/packout	把多个变量打包为一个大变量，以减少拷贝数据到局存的操作次数。
swap/swapin/swapout	对数组进行转置并拷贝到局存，以消除不连续的访问。
swapahead/swapback/swapalloc	仅进行数组转置，不拷贝到局存。用在非结构数据区上，提前进行数组转置，之后配合 copy 系列子句拷贝转置后数据到局存
num_gangs/num_workers	指定加速线程的 gang 和 worker 的数目
gang/worker	将循环映射到 gang 或 worker 上执行
present	指定相关数据已在设备内存中
tile	将循环按指定的块大小分割成两重循环
collapse	指定有多少层紧嵌套的循环跟 loop 指示相关联，这些循环的所有迭代将被统一调度
annotate	对循环映射、数组大小、数组属性(如只读、访问特点)等进行补充说明
async/wait	用于操作的异步执行和等待。

注：本文档中所描述的除 **parallel**、**enter data**、**exit data**、**update**、**schedyield** 之外的指示（非子句），均需要在 **parallel**、**parallel loop** 作用范围内使用，除非有特殊说明。

3.4 加速计算区指示 **parallel**

加速计算区编译指示定义了一个加速计算区，编译器会将其封装成加速任务，加速任务（加速计算区内的代码）将加载到加速设备上运行。

1) 语法

C/C++	<pre>#pragma acc parallel [子句列表] structured-block</pre>
C/C++	
Fortran	<pre>!\$ACC PARALLEL [子句列表] structured-block [\$ACC END PARALLEL]</pre>
Fortran	

子句：

```

async [(整数表达式)]
wait [(整数表达式列表)]
num_gangs(整数表达式)
num_workers (整数表达式)
if (条件语句);
reduction (操作符: 变量列表);
dis_reduction (操作符: 变量列表);
copy (变量列表);
copyin (变量列表);
copyout (变量列表);
local (变量列表);
pack(变量列表);
packin (变量列表);
packout(变量列表);
swap(变量列表);
swapin(变量列表);
swapout(变量列表);
private (变量列表);
firstprivate (变量列表);
cache (变量列表);
present(变量列表);
uncache(变量列表);
annotate (暗示子句)。

```

2) 说明

当主进程遇到并行结构并且下列情况为真时，它就创建若干个 **gang** 来执行这个加速器并行区域，而每个 **gang** 又包含若干个 **worker**：

- a) 没有 **if** 子句的出现；
- b) **if** 中条件表达式值非 0(C/C++)或为.true.(Fortran)。

创建的这些 **gang** 一旦创建，**gang** 的数量和每个 **gang** 包含的 **worker** 数量在整个并行区域内都保持不变。接着，每个 **gang** 中的所有 **worker** 都开始执行构件中结构块的代码。**gang** 的数量由 **num_gangs** 指定，每个 **gang** 包含的 **worker** 数量由 **num_workers** 指定。**num_gangs** 和 **num_workers** 的乘积将是执行该加速任务（加速并行区）的线程数。如不指定 **num_gangs** 和 **num_workers**，将由实现时决定实际运行的参数。

由于实现时的需要，要求 **num_gangs** 和 **num_workers** 的乘积是 8 的倍数，并且该乘积要小于等于作业提交时指定启动的总线程数。可以根据并行区的计算负载情况，合理设置 **num_gangs** 和 **num_workers**，在有多个无数据相关性加速任务时，配合使用 **async** 子句，可以实现多个无相关任务的并行执行。

主进程创建好加速线程组后，不参与加速区域的执行，默认将等待加速线程组的执行结束。如果 **if** 表达式的值是 0，那么这个加速区由主进程串行执行。

加速线程组一旦创建，运行时系统不会改变线程组中的线程数，并在加速区执行期间保持不变。加速区动态范围中的语句由每个加速线程执行，并且每个加速线程可以执行与其他加速线程不同的语句轨迹。加速区的结尾隐含有一个栅栏。

如果有 **async** 子句，主进程在完成加速线程组的创建后，不等待加速线程组执行的结束，而直接执行后续代码，需要通过 **wait** 机制在后续代码中等待该加速任务的完成。

加速区存在以下限制：

- a) 加速计算区指示 `parallel` 不支持嵌套；
- b) 不能有进出加速区的其他分支，否则可能会导致不确定的错误；
- c) 程序的正确性不能依赖于 `parallel` 编译指示的子句出现的顺序；
- d) `if` 子句最多只能出现一次；
- e) 在加速区中不能访问私有属性的数据，比如 Fortran 程序中 `module` 中定义的 `private` 变量。

3) 示例

```

1 /*
2  * FILE:ex_3_4.c
3  */
4 #include <stdio.h>
5
6 main()
7 {
8     #pragma acc parallel
9     {
10         printf("hello world\n");
11     }
12
13     return 0;
14 }

```

图 3-2 编译指示 `parallel` 的程序示例

图 3-2 中定义了一个加速区，加速区中的代码将被所有加速线程执行。

3.4.1 if 子句

1) 语法

```
if(条件语句)
```

2) 说明

`if` 子语是 `parallel` 指示的可选项；没有 `if` 子语的时候，编译器为本区域生成加速器上执行的代码。

有 `if` 子语的时候，编译器为本区域生成两份代码，一份可以在加速器上执行，另一份可以在主机执行。对 C 和 C++ 语言，`if` 子语中的条件值为零时，对 Fortran 语言，`if` 子语中的条件值为 `.false.` 时，主机端代码将被执行。`if` 子语中条件的值在 C 和 C++ 语言中为非零时，或在 Fortran 语言中为 `.true.` 时，加速器端代码将被执行。

(3) 示例

```

1 /*
2  * FILE:ex_3_4_1-01.c
3  */
4 #include <stdio.h>
5
6 main()
7 {
8     int count=0;
9
10    #pragma acc parallel if(count == 0)

```

```

11  {
12      count ++;
13  }
14
15  printf("count=%d\n",count);
16
17  return 0;
18 }

```

图 3-3 if 子句 C 程序示例

```

1 !FILE:ex_3_4_1-02.f90
2
3 program main
4     implicit none
5     integer count
6     count=0
7
8 !$ACC PARALLEL if(count .eq. 0)
9     count = count + 1
10 !$ACC END PARALLEL
11     print *, "count=",count
12
13 end

```

图 3-4 if 子句 Fortran 程序示例

3.4.2 私有化子句 **private/firstprivate**

参见 3.17.1 [P54]。

3.4.3 本地化子句 **local**

参见 3.17.2 [P56]。

3.4.4 规约子句 **reduction**

参见 3.17.3 [P56]。

3.4.5 缓存子句 **cache**

1) 语法

```
cache(变量列表)
```

2) 说明

cache 子句指明需尽可能将变量列表中的变量放到最快的内存中（如加速设备局存），使用最有效的存取方式进行数据交互，具体实现由编译系统实现时决定。变量列表中指定的变量以逗号分隔；变量之后可通过冒号(:)指定其缓存大小(单位为 1 个 cache 行，系统默认为 2 个 cache 行)，例如 p:8，表示 p 占用 8 个 cache 行（其中一个 cache

行为 256 字节)。cache 变量的访问一致性由用户程序保证，通常用于只读变量或私有变量。

3) 使用注意事项

- a) 使用 **cache** 子句的程序，运行时需要使用 **-b** 选项；
- b) **cache** 子句中的变量类型只能是基本数据类型，不支持自定义类型和派生数据类型
- c) 用户程序保证 **cache** 变量的起始地址与长度都按 **256B** 对界。例外的情况是，用户程序可以不用考虑先私有化再使用 **cache** 的变量对界要求；
- d) Fortran 程序可使用 **!\$acc align(varlist)** 指示进行变量对界，按 **256B** 对界；对界指示的变量如果出现在 **common** 段中，只保证 **common** 段起始地址按 **256B** 对界；
- e) C 程序变量对界用法参照基础编译手册，在变量定义后添加属性 **__attribute__((aligned(256)))**；
- f) 对于需要修改内容的 **cache** 变量，不能有通过别名进行的访问；
- g) **cache** 作用范围内不能对 **cache** 变量取地址；
- h) **cache** 只作用于数组；
- i) 对于 C 程序，栈数组不能使用 **cache**；
- j) 一般是在无法使用 **copy** 机制的时候使用 **cache**；
- k) 只读数组使用 **cache** 子句时，可以配合 **annotate(readonly(varlist))**，标记对应的变量为只读属性，可优化 **cache** 子句的性能。

4) 示例

```

1 /*
2  * FILE:ex_3_4_5.c
3  */
4
5 #include <stdio.h>
6 #define M      1024
7 #define N      512
8 #define NUM    1024
9
10 int colidx[NUM];
11 int a[NUM];
12 int p[N];
13 long w[M+1];
14 int rowstr[M+1];
15
16 main()
17 {
18     int i,j,k;
19     int firstrow,lastrow;
20     long sum;
21
22     firstrow=1;
23     lastrow=M+1;
24     for(i=0;i<M+1;i++)
25         rowstr[i]=i;
26     for(i=0;i<NUM;i++)
27     {
28         a[i]=i;
29         colidx[i]=i%N;

```

```

30     }
31     for(i=0;i<N;i++)
32         p[i]=i;
33
34 #pragma acc parallel loop copyin(rowstr,a) copyout(w) \
35         cache(p:8,colidx) annotate(entire(a))
36     {
37         sum = 0.0;
38         for (k = rowstr[j]; k < rowstr[j+1]; k++)
39         {
40             sum = sum + a[k]*p[colidx[k]];
41         }
42         w[j] = sum;
43     }
44
45     return 0;
46 }

```

图 3-5 cache 子句示例

在图 3-5 中，因对数组 colidx 的访问是动态的，对数组 p 的访问是通过间接下标进行的，无法使用规则的 copy 语句，使用 cache 指示后，编译器将视情况选择尽可能快的内存来存放这两个数组。

3.4.6 数据拷贝子句 copy/copyin/copyout

参见 3.17.4 [P59]。

3.4.7 数据打包子句 pack/packin/packout

对应数据传输而言，零散变量的传输效率比批量的数据拷贝效率要差，因此把零散变量打包为一个大的变量可减少数据拷贝的开销，数据打包子句与数据拷贝子句类似。

1) 语法

```

pack(变量列表[,at data index])
packin(变量列表[,at data index])
packout(变量列表[,at data index])

```

子句含义：

- a) pack 子句：先打包，然后操作步骤同 copy 子句，之后解包；
- b) packin 子句：先打包，然后操作步骤同 copyin 子句；
- c) packout 子句：首先操作步骤同 copyout 子句，之后解包。

2) 说明

参数含义如下：

- a) 变量列表以逗号分隔，支持标量、数组等。at data index 用于指定打包操作在程序中的位置，称之为打包点（其中 index 是常数，与 data 指示的 index(num) 的 num 对应），OpenACC*支持打包点的设置，如果不指定打包点，编译器默认在 data/parallel 的前后分别插入打包解包语句。如果用户指定打包点，编译器就在指定的 data 前后分别进行打包和解包；指定打包点时需要注意，打包点的作用是将数据打包的操作放到打包点处进行操作，因此要保证在打包点和

当前位置之间相关的数据不会发生改变，否则会导致结果错。打包的变量只支持变量和数组指针，不支持普通指针。

- b) 默认情况下，pack/packin/packout 会根据循环的划分方式将对应的数据进行分割。如果用户使用暗示语句指出数据需要全部拷入，packin 会将对应的数据整个拷贝到加速设备内存上，一般用于只读数据

3) 示例

```

1 /*
2  * FILE:ex_3_4_7-01.c
3  */
4
5 #include <stdio.h>
6 #include <stdlib.h>
7
8 #define N      128
9
10 void init(int *A, int num,int *x,int *y,int *z)
11 {
12     int i;
13
14     for(i=0;i<num;i++)
15         A[i]=i;
16
17     *x=rand()%64;
18     *y=rand()%64;
19     *z=rand()%64;
20 }
21
22 main()
23 {
24     int i;
25     int A[N];
26     int x,y,z;
27     int t;
28
29     init(A,N,&x,&y,&z);
30
31     #pragma acc parallel loop packin(x,y,z) copyin(A)  annotate(entire(A))
32     for (i = 0; i < N; i++)
33     {
34         t=A[x+y+z];
35     }
36
37     printf("t=%d \n", t);
38
39     return 0;
40 }

```

图 3-6 数据打包语句例子 1

在图 3-6 中的程序将标量 x、y、z 打包为一个大的变量，然后通过一次数据拷贝操作到每个加速线程的设备内存中，打包的语句插在加速段之前。

```

1 /*
2  * FILE:ex_3_4_7-02.c

```

```

3  */
4
5  #include <stdio.h>
6  #include <stdlib.h>
7
8  #define N      128
9
10 void init(int *A, int num)
11 {
12     int i;
13
14     for(i=0;i<num;i++)
15         A[i]=i;
16 }
17
18 main()
19 {
20     int i,j,k;
21     int A[N],B[N],C[N];
22
23     init(A,N);
24     init(B,N);
25
26     #pragma acc data index(1)
27     {
28         for(k=0;k<N;k++)
29         {
30             for(j=0;j<N;j++)
31             {
32                 #pragma acc parallel loop packin(A,B,at data 1) copyout(C)
33                 for (i = 0; i < N; i++)
34                 {
35                     C[i] = A[i] + B[i];
36                 }
37             }
38         }
39     }
40
41     return 0;
42 }

```

图 3-7 数据打包子句例子 2

在图 3-7 中，程序通过 `at data 1` 指定了数组 A、B 打包操作的数据点（或称为打包点），其中的数字 1 与 `data` 指示中的 `index(1)` 相对应，即数组 A、B 打包的操作实际上是在 `data` 指示处进行。由于数组 A、B 在循环内是只读的，因此可以将打包操作提前到循环之外，这样可以减少打包/解包操作的次数，避免不必要的开销。由于数组 C 在循环内被写覆盖，所以只需要 `copyout`，而不需要和数组 A、B 一起打包。

3.4.8 数组转置子句 `swap/swapin/swapout`

对访问方式不连续的数组的拷贝，会导致编译器生成带跨步的数据拷贝操作，性能相对较差，用户可以使用转置编译指示来标识这种数组。编译器将数组转置后，可通过连续的数据拷贝操作进行拷贝，以提高数据拷贝的性能。

1) 语法

```

swap(数组名((dimension order:,,,....),....[at data index])
swapin(数组名((dimension order:,,,....),....[at data index])
swapout(数组名((dimension order:,,,....),....[at data index])

```

子句含义:

- a) swap 子句: 先将数组进行转置操作, 然后操作步骤同 copy 子句一致, 之后再转置回原数组;
- b) swapin 子句: 先将数组进行转置操作, 然后操作步骤同 copyin 子句一致;
- c) swapout 子句: 先将程序中的数组访问替换成对转置后的数组访问, 然后操作步骤同 copyout 子句, 再转置回原数组。

2) 说明

默认情况下, swap/swapin/swapout 会根据循环的划分方式将对应的数据进行分割, 如果用户用暗示语句指出数据需要全部拷入, swapin 会将对应的数据整个拷贝到加速设备内存上, 一般用于只读数据。

OpenACC*支持转置点的设置, 如果不指定转置点, 编译器默认在 data/parallel 标识的代码区域的开始或结束位置插入转置或反转置语句。如果用户指定转置点, 编译器就在指定的 data 标识的代码区域开始或结束位置进行转置或反转置。指定转置点时需要注意, 转置点的作用是将数据转置的操作放到转置点处进行操作, 因此要保证在转置点和当前位置之间相关的数据不会发生改变, 否则会导致结果错。

注意: 转置功能需要在主存中申请存放转置后数据的空间, 如内存空间无法满足需求请谨慎使用。

维度次序 (dimension order), 标识数组维度的编号, 从低维到高维递增:

- a) Fortran 程序的数组维度按从左到右编号: 1, 2, 3, 4, ... 其中 1 表示最低维; 描述数组维度顺序也是从左到右排列, 即从低维到高维, 如三维数组的维度顺序是(1,2,3);
- b) C 程序的数组维度按从右到左编号: 1, 2, 3, 4, ... 描述数组维度次序时按照从右到左排列, 即从低维到高维, 如三位数组的维度顺序是(3,2,1)。

3) 示例

```

1 /*
2  * FILE:ex_3_4_8.c
3  */
4
5 #include <stdio.h>
6
7
8 float FDV[32][64][128];
9 float FDV1k,FDV1[64],FDV0T[64];
10
11 extern float FDV[32][64][128];
12 extern float FDV1k,FDV1[64],FDV0T[64];
13
14 void test_swap(int im,int jm, int km)
15 {
16     int i, j, k;
17
18 #pragma acc parallel loop swapin(FDV(dimension order:2,1,3))
19     for(j = 0; j < jm; j++)
20         for(i = 0; i < im; i++)

```

```

21         for(k = 0;k < km;k ++)
22         {
23             FDV1k=FDV[k][j][i];
24             FDV1[k]=FDV1k;
25             FDV0T[k]=FDV1k;
26         }
27     }
28
29 main()
30 {
31     test_swap(128,64,32);
32
33     return 0;
34 }

```

图 3-8 数组转置子句示例

在图 3-8 的例子中,变化最快的内层循环 k 对应数组 FDV 的最高维,也就是对 FDV 的访问是不连续的,这种访问方式在多核系统中,其 cache 命中率也是很低的,效率不高。其中 swapin 的作用是对 FDV 数组进行转置,FDV 数组的维度次序是(3,2,1),转置后新数组的维度顺序是(2,1,3),如 FDV_new[64][128][32]。同时程序中对 FDV 的访问(FDV[k][j][i])将会被替换成对转置后数组 FDV_new 的访问(FDV_new[j][i][k])。转置后对 FDV_new 的访问是连续的,可以较大的改善程序性能。

3.4.9 num_gangs 子句

1) 语法

```
num_gangs_(整数表达式)
```

2) 说明

parallel 构件允许使用 num_gangs 子语。整数表达式的值规定并行执行该区域的 gang 的数量。如果没有本子语,编译器将使用自定义的默认值。

3) 示例

```

1  /*
2   * FILE:ex_3_4_9.c
3   */
4
5  #include <stdio.h>
6  #define N      1024
7  int a[N][N];
8
9  main()
10 {
11     int i,j;
12     int err_flg;
13
14     int n=32;
15     #pragma acc parallel num_gangs(n) num_workers(2)
16     #pragma acc loop gang
17         for(i=0;i<N;i++)
18     #pragma acc loop worker
19         for(j=0;j<N;j++)
20     {

```

```

21         a[i][j]=i*N+j;
22     }
23
24
25     for(i=0;i<N;i++)
26         for(j=0;j<N;j++)
27             if(a[i][j]!=i*N+j)
28                 err_flg++;
29     if(err_flg)
30         printf("test FAILED!\n");
31     else
32         printf("test OK!\n");
33
34     return 0;
35 }

```

图 3-9 num_gangs 子句示例

图 3-9 的例子中设定了并行执行该区域的 gang 的数量是 32。

3.4.10 num_workers 子句

1) 语法

```
num_works_(整数表达式)
```

2) 说明

parallel 构件允许使用 num_workers 子语。整数表达式的值规定了执行该区域的每个 gang 中所包含 worker 的数量。如果没有本子语，编译器将使用自定义的默认值；默认值可能是 1。

3) 示例

```

1 /*
2  * FILE:ex_3_4_10.c
3  */
4
5 #include <stdio.h>
6 #define N      1024
7 int a[N][N];
8
9 main()
10 {
11     int i,j;
12     int err_flg;
13
14     int n=4;
15 #pragma acc parallel num_gangs(n) num_workers(16)
16 #pragma acc loop gang
17     for(i=0;i<N;i++)
18 #pragma acc loop worker
19         for(j=0;j<N;j++)
20             {
21                 a[i][j]=i*N+j;
22             }
23
24

```

```

25     for(i=0;i<N;i++)
26         for(j=0;j<N;j++)
27             if(a[i][j]!=i*N+j)
28                 err_flg++;
29     if(err_flg)
30         printf("test FAILED!\n");
31     else
32         printf("test OK!\n");
33
34     return 0;
35 }

```

图 3-10 num_workers 子句示例

图 3-10 的例子中设定了并行执行该区域的每个 gang 中 worker 的数量是 16。

3.4.11 异步子句 **async**

参见 3.17.5 [P63]。

3.4.12 等待子句 **wait**

参见 3.17.6 [P65]。

3.4.13 数据重用子句 **present**

参见 3.17.7 [P66]。

3.4.14 分布规约子句 **dis_reduction**

参见 3.17.8 [P67]。

3.4.15 不可缓存子句 **uncache**

参见 3.17.9 [P68]。

3.4.16 暗示子句 **annotate**

参见 3.17.10 [P68]。

3.5 循环映射指示 **loop**

编译指示 **loop** 作用于紧跟其后的循环，主要用于描述执行该循环所采用的并行方式等属性。

1) 语法

C/C++

```
#pragma acc loop [子句列表]
for 循环
```

C/C++

Fortran

```
!$ACC LOOP [子句列表]
do 循环
[$ACC END LOOP]
```

Fortran

其中的子句是下列之一：

collapse(正整数常数);
gang;
worker;
vector;
tile (整数表达式列表) ;
private(变量列表) ;
reduction (操作符: 变量列表);
annotate (暗示子句)。

2) 说明

OpenACC*要求加速循环必须是规范的，对循环语句有如下限制(假设循环变量为 i):

- a) 初值表达式：只支持形式 $i = \text{expr}$;
- b) 条件表达式：只支持 $i < \text{expr}$ 、 $i \leq \text{expr}$ ，其余均无效，结果不保证正确；
- c) 步进表达式：只支持形式为 $i += 1$ 、 $i++$ ，其余结果不保证正确；
- d) 循环中不能有中断该循环的语句，如 **break**，跳出循环的 **goto** 等。
- e) 循环变量的值除了在步进表达式中被修改之外，在循环执行过程中不能修改循环变量的值，否则程序运行可能会出错。在循环执行结束后循环变量中的值是不确定的。

loop 指示需要在 **parallel** 的动态范围内使用，否则将不产生实际作用。需要并行执行的 **loop** 指示必须出现在 **parallel** 的语法范围内，即出现在被 **parallel** 加速计算区调用的函数内的 **loop** 指示只能是串行执行。在 **parallel** 内，最多只允许出现一个与 **parallel** 嵌套的 **loop** 指示，不支持 **parallel** 内嵌套多个并列的 **loop** 指示，**loop** 指示间的嵌套不受此限，否则有可能导致错误的结果。通常情况下 **parallel** 和 **loop** 需要紧嵌套使用。

当 **parallel** 中的 **loop** 指示均没有指定 **gang**、**worker** 子句时，默认情况下最外层的 **loop** 指示作用的循环将在多个 **gang** 间并行执行，**gang** 的数目等于加速线程的数目，每个 **gang** 中只有一个 **worker**。如果不需要多级并行，则可以不指定 **gang**、**worker** 子句。

loop 指示的执行方式有四种，按照从高到低的层次分别为 **gang**、**worker**、串行、**vector**，当多个 **loop** 指示嵌套时，需要注意执行方式的嵌套层次。

当 **loop** 指示出现在 **parallel** 加速计算区所调用的函数中时，实际有效的子句仅为 **vector**，其他子句将被忽略。

3) 示例

```
1 /*
2  * FILE:ex_3_5.c
3  */
4
5 #include <stdio.h>
```

```

6 #include <stdlib.h>
7 #include <math.h>
8
9 #define N      1024
10 double a[N],r[N];
11
12 void init()
13 {
14     int i;
15
16     for(i=0;i<N;i++)
17     {
18         a[i]=((double)(rand()%N))/N;
19     }
20 }
21
22 main()
23 {
24     int i;
25     double s,c;
26     int n;
27     n=N;
28     double result;
29
30     #pragma acc parallel copyin(a) copyout(r)
31     {
32         #pragma acc loop
33         for( i = 0; i < n; ++i )
34         {
35             s = sin(a[i]);
36             c = cos(a[i]);
37             r[i] = s*s + c*c;
38         }
39     }
40
41     result=0.0;
42     for(i=0;i<n;i++)
43         result += r[i];
44
45     printf("result=%lf\n", result);
46
47     return 0;
48
49 }

```

图 3-11 编译指示 loop 的程序示例

3.5.1 gang 子句

1) 语法

```
gang
```

2) 说明

在一个 parallel 加速区中，gang 子语要求将关联的循环中的迭代步分摊到 parallel 指示创建的多个 gang 上，从而实现并行执行。循环的所有迭代步必须没有相

关性，但在 `reduction` 子语中指定的变量除外。带有 `gang` 子句的 `loop` 指示内不允许出现带有 `gang` 子句的 `loop` 指示。

3) 示例

通常 `gang` 子句和 `worker` 子句配合使用来实现多级循环划分，示例见图 3-12。

3.5.2 worker 子句

1) 语法

```
worker
```

2) 说明

在一个加速器 `parallel` 区域内，`worker` 子语指明，关联循环的迭代步将被分散给单个 `gang` 的多个 `worker` 并行执行。循环的所有迭代步必须是数据独立的，但 `reduction` 子句中指定的变量除外。带有 `worker` 子句的 `loop` 指示中不允许出现 `gang`、`worker` 子句的 `loop` 指示。

3) 示例

```
1 /*
2  * FILE:ex_3_5_2.c
3  */
4
5 #include<stdio.h>
6 #include <stdlib.h>
7 #define N 100
8 int A[N][N];
9
10 main()
11 {
12     int i,j;
13     #pragma acc parallel num_gangs(8) num_workers(2)
14     {
15         #pragma acc loop gang
16         for( i = 0; i < N; i++ )
17             #pragma acc loop worker
18             for( j = 0; j < N; j++ )
19             {
20                 A[i][j]= i+j;
21             }
22     }
23     for( i = 0; i < N; i++ )
24         for( j = 0; j < N; j++ )
25         {
26             if(A[i][j]!=i+j)
27             {
28                 printf("test fail!\n");
29                 exit(0);
30             }
31         }
32
33     printf("test ok!\n");
34
35     return 0;
36 }
```

图 3-12 worker 子句示例

例中表示将 j 层 for 循环的迭代步分摊到每个 gang 里面所包含的两个 worker 上面，从而实现并行执行。

3.5.3 向量化 vector 子句

1) 语法

vector

2) 说明

提示编译器对指定的循环进行 SIMD 向量化并行。一般用于串行执行的循环。

该功能暂时未支持。

3) 示例

```

1 /*
2  * FILE:ex_3_5_3.c
3  */
4
5 #include<stdio.h>
6 #include <stdlib.h>
7
8 #define N 64
9 int A[N][N][N];
10
11 main()
12 {
13     int i,j,k;
14
15     #pragma acc parallel num_gangs(8) num_workers(2)
16     {
17         #pragma acc loop gang
18         for( i = 0; i < N; ++i )
19             #pragma acc loop worker
20             for( j = 0; j < N; ++j)
21                 #pragma acc loop vector
22                 for( k = 0; k < N; ++k)
23                 {
24                     A[i][j][k]= i+j+k;
25                 }
26     }
27     for( i = 0; i < N; i++ )
28         for( j = 0; j < N; j++ )
29             for(k = 0; k < N; k++ )
30             {
31                 if(A[i][j][k]!=i+j+k)
32                 {
33                     printf("test fail!\n");
34                     exit(0);
35                 }
36             }
37     printf("test ok!\n");
38
39     return 0;
40 }

```

图 3-13 vector 子句示例

例中表示对最内层的串行执行的 for 循环进行 SIMD 向量化并行。

3.5.4 循环合并子句 collapse

1) 语法

```
collapse(正整数常数)
```

2) 说明

collapse 子句用于指定有多少层紧嵌套的循环与 loop 指示关联，collapse 的参数必须是一个常量正整数，如果没有 collapse 子句，loop 指示只作用于紧跟的循环。与 collapse 关联的所有循环的步长必须是可计算的，并且在循环执行过程中保持不变。

3) 示例

```

1 /*
2  * FILE:ex_3_5_4.c
3  */
4
5 #include<stdio.h>
6 #include <stdlib.h>
7
8 #define N 100
9 int A[N][N];
10 main()
11 {
12     int i,j;
13
14     #pragma acc parallel
15     {
16         #pragma acc loop collapse(2)
17         for( i = 0; i < N; i++ )
18             for( j = 0; j < N; j++ )
19             {
20                 A[i][j]= i+j;
21             }
22     }
23     for( i = 0; i < N; i++ )
24         for( j = 0; j < N; j++ )
25         {
26             if(A[i][j]!=i+j)
27             {
28                 printf("test fail!\n");
29                 exit(0);
30             }
31         }
32     printf("test ok!\n");
33
34     return 0;
35 }
```

图 3-14 collapse 子句示例

例中表示将 parallel 并行区中的 i、j 两层循环合并。

3.5.5 循环分块子句 tile

1) 语法

```
tile(块大小)
tile(块大小列表)
tile(循环迭代变量 i:块大小 1, 循环迭代变量 j:块大小 2, ...)
```

2) 说明

块大小为正整数常数，块大小列表由多个块大小组成，中间以逗号分隔，tile 子句有三种具体的用法。

用法 1: tile(块大小)，仅作用于当前 loop 指示，表示 loop 指示后紧跟的循环将以该块大小进行分割。

用法 2: tile(块大小列表)，当列表中有 n 个参数时，则要求 loop 指示后有 n 个紧嵌套的循环，列表中第一个参数作用于最内层循环，最后一个参数作用于最外层循环，依次类推。

用法 3: tile(循环迭代变量 i:块大小 1, 循环迭代变量 j:块大小 2, ...)，作用于 loop 指示内直接嵌套的所有循环，与列表中循环变量相同的循环使用对应的块大小进行分割，比如循环迭代变量是 i 的循环，均以块大小 1 进行分割，以此类推。用法 3 是对用法 1 的补充，当存在多个相同循环变量的循环时，使用方式 3 较为简便。需要注意的是，当同时以不同的方式对同一个循环设置块大小，且块大小不一致时，由实现时决定。

3) 示例

```
1 /*
2  * FILE:ex_3_5_5-01.c
3  */
4
5 #include<stdio.h>
6 #include<stdlib.h>
7 #define M 64
8 #define N 4096
9
10 int A[M][N];
11
12 main()
13 {
14     int i, j;
15
16     #pragma acc parallel copyout(A)
17     #pragma acc loop
18     for(i = 0; i < M; i++)
19     {
20         #pragma acc loop tile(1024)
21         for(j = 0; j < N; j++)
22             A[i][j] = i*N+j;
23     }
```

```
1
2 /*
3  * FILE:ex_3_5_5-02.c
4  */
5
6 #include<stdio.h>
7 #include<stdlib.h>
8 #define M 64
9 #define N 4096
10
11 int A[M][N];
12
13 main()
14 {
15     int i, j;
16
17     #pragma acc parallel copyout(A)
18     #pragma acc loop tile(i:1, j:1024)
19     for(i = 0; i < M; i++)
20         for(j = 0; j < N; j++)
21             A[i][j] = i*N+j;
22
23     for(i=0;i<M;i++)
```

24		24	for(j=0;j<N;j++)
25	for(i=0;i<M;i++)	25	{
26	for(j=0;j<N;j++)	26	if(A[i][j]!=i*N+j)
27	{	27	{
28	if(A[i][j]!=i*N+j)	28	printf("Test fail!\n");
29	{	29	exit(0);
30	printf("Test fail!\n");	30	}
31	exit(0);	31	}
32	}	32	printf("Test OK!\n");
33	}	33	
34	printf("Test OK!\n");	34	return 0;
35		35 }	
36	return 0;		
37 }			

图 3-15 tile 子句示例

tile 子句使用示例如上图所示，左侧代码为使用方式 1 对 i 和 j 循环分块的使用法，右侧代码是通过用法 3 实现同样效果的代码。

3.5.6 私有化子句 private

参见 3.17.1.1 [P54]。

3.5.7 规约子句 reduction

参见 3.17.3 [P56]。

3.5.8 分布规约子句 dis_reduction

参见 3.17.8 [P67]。

3.5.9 暗示子句 annotate

参见 3.17.10 [P68]。

3.6 加速数据区指示 data

加速数据区编译指示 data 用于定义加速计算的数据区（简称加速数据区），可以用来描述变量在加速设备内的属性。

1) 语法

C/C++	
#pragma acc data	子句[子句列表]
structured-block	

	C/C++
	Fortran
!\$ACC DATA 子句[子句列表] structured-block !\$ACC END DATA	
	Fortran

其中，子句列表是可选项，可根据实际需要设置子句列表；子句列表中的子句以空格分隔。

子句可为下列之一：

if(条件表达式);
copy(变量列表);
copyin(变量列表);
copyout(变量列表);
local(变量列表);
present(变量列表);
async[(整数表达式)]。

2) 说明

data 指示在加速计算区 **parallel** 之内使用有效，用于描述主存与设备内存间的数据拷贝操作，**data** 指示可以嵌套使用。**data** 上数据拷贝的有效范围是当前 **data** 的作用域，超出该作用域后，设备内存上的相关拷贝数据不保证继续有效。

3.6.1 if 子句

1) 语法

```
if (条件表达式)
```

2) 说明

if 子句是可选的子句。

如果没有出现 **if** 子句，编译器将根据编译指示内容对编译指示作用区域内的代码进行变换，例如在代码区域前面申请局存空间并插入从主存拷入数据、在代码区域结束处拷出数据到主存的代码。

如果出现了 **if** 子句，编译器将根据 **if** 子句的条件表达式的值来确定是否对编译指示作用区域内的代码进行变换。当 **if** 子句的条件表达式的值为非 0（C 与 C++）或 **true**.（fortran）时，编译器将根据编译指示内容对编译指示作用区域内的代码进行变换；当 **if** 子句的条件表达式的值为 0（C 与 C++）或 **false**.（fortran）时，编译器将不对编译指示作用区域内的代码进行变换。

3) 示例

if 子句的例子 1：图 3-18 所示例子测试在 **data** 编译指示上使用 **if** 子句。

```

1 !FILE:ex_3_6_1.f90
2
3 function fun(i)
4     integer i, fun
5     fun = i + 1
6     if(fun == 129) then
7         fun =1
8     endif
9 end

```

```

10
11 program main
12     implicit none
13     integer, external :: fun
14     integer flag
15     integer A(128,128), b(128,128),m,i,j,x
16
17     do i=1,128
18         do j=1,128
19             A(j,i) = i + j
20         enddo
21     enddo
22
23 !$ACC PARALLEL LOOP LoCaL(m,flag,j) copyout(B) tile(2)
24     do i=1,128
25         m = fun(i)
26         flag=mod(m-1,2)
27 !$ACC DATA COPYIN(A(*, m)) if(flag .eq. 1)
28         do j=1,128
29             B(j, i) = A(j, m)
30         enddo
31 !$ACC END DATA
32     enddo
33 !$ACC END PARALLEL LOOP
34     do i=1,128
35         do j=1,128
36             if(A(j, i) /= i + j) then
37                 print *, "TEST FAIL.A"
38                 stop
39             endif
40         enddo
41     enddo
42     do i=1,128
43         m = fun(i)
44         flag = mod((m-1),2)
45         do j=1,128
46             if(flag == 1) then
47                 if(B(j, i) /= A(j, m)) then
48                     print *, "TEST FAIL.B"
49                     stop
50                 endif
51             else
52                 x = m-1;
53                 if(x == 0) then
54                     x=128
55                 endif
56                 if(B(j, i) /= A(j, x)) then
57                     print *, "TEST FAIL.B"
58                     stop
59                 endif
60             endif
61         enddo
62     enddo
63
64     print *, "TEST OK"
65
66 end

```

图 3-16 编译指示 data 上 if 子句的程序示例

3.6.2 数据拷贝子句 **copy/copyin/copyout**

参见 3.17.4 [P59]。

3.6.3 数据重用子句 **present**

参见 3.17.7 [P66]。

3.6.4 本地化子句 **local**

参见 3.17.2 [P56]。

3.6.5 异步子句 **async**

参见 3.17.5 [P63]。

3.6.6 **index** 子句

1) 语法

```
index(num)
```

2) 说明

当 data 编译指示在 parallel 编译指示的作用域之外使用时,实际有效的子句为 index。此处 index 子句的含义是将当前 data 编译指示所在位置编号为 num, 其中 num 的值必须是整数 (≥ 0), 便于进行程序中数据操作点的设置 (包括设置数据打包点和数据转置点)。具体用法还可参考“3.4.7 数据打包子句[P22]”和“3.4.8 数组转置子句[P24]”。

3) 示例

```
1 /*
2  * FILE:ex_3_6_6.c
3  */
4
5 #include <stdio.h>
6 #include <stdlib.h>
7
8 #define NN 128
9
10 int A[NN], B[NN], C[NN];
11
12 int main()
13 {
14     int i;
15
16     for(i = 0; i < NN; i++)
17     {
```

```

18     A[i] = 1;
19     B[i] = 2;
20 }
21
22 #pragma acc data index(1)
23 {
24     #pragma acc parallel loop packin(A, B, at data 1) copyout(C)
25     for(i = 0; i < NN; i++)
26     {
27         C[i] = A[i] + B[i];
28     }
29 }
30
31 for(i = 0; i < NN; i++)
32 {
33     if(C[i] != 3)
34     {
35         printf("Test Error! C[%d] = %d\n", i, C[i]);
36         exit(-1);
37     }
38 }
39
40 printf("Test OK!\n");
41 return 0;
42 }

```

图 3-17 编译指示 data 上 index 子句的程序示例

3.6.7 数据分块子句 block

1) 语法

```
block(循环迭代变量:块大小)
```

2) 说明

block 子句完成加速数据区指示 data 上拷贝数组的循环迭代变量所指示的维度的数据分块，这里的数据分块将会引起相应层次循环上的循环分块。目前 block 中的循环迭代变量必须是 data 指示所作用的最外层循环的迭代变量，不支持内层循环的分块。循环分块以后，该分块的外层循环将会被外提至 parallel 作用区域的最外层，如果不需要此分块外提功能，可以通过选项-noLoopOuter 关闭。

3) 示例

```

1 /*
2  * FILE:ex_3_6_7-stream14.c
3  */
4
5 #include <stdio.h>
6 #define NX 512
7 #define NY 128
8 #define NZ 64
9 #define NN 32
10
11 int AA[NX][NY][NZ][NN];
12 int BB[NX][NY][NZ][NN];
13 int BB1[NX][NY][NZ][NN];

```

```

14
15 void main()
16 {
17
18     int i, j, k, l, m;
19
20     for(i=0; i<NX; i++)
21         for(j=0; j<NY; j++)
22             for(k=0; k<NZ; k++)
23                 for(l=0; l<NN; l++)
24                     AA[i][j][k][l] = i+j+k+l;
25
26
27 //HOST time:
28     for(i=1; i<NX-1; i++)
29     {
30         for(j=1; j<NY-1; j++)
31         {
32             for(k=1; k<NZ-1; k++)
33                 for(l=1; l<NN-1; l++){
34                     BB1[i][j][k][l] = AA[i][j][k][l] ;
35                 }
36         }
37     }
38
39
40 //ACC time:
41 #pragma acc parallel loop
42     for(i=1; i<NX-1; i++)
43     {
44         #pragma acc loop tile(5)
45         for(j=1; j<NY-1; j++)
46         {
47             #pragma acc data copyin(AA[i][j][*][*]) copyout(BB[i][j][*][*])
48             block(k:10)
49             {
50                 for(k=1; k<NZ-1; k++)
51                     for(l=1; l<NN-1; l++){
52                         BB[i][j][k][l] = AA[i][j][k][l];
53                     }
54             }
55         }
56     }
57
58
59     for(i=1; i<NX-1; i++)
60     {
61         for(j=1; j<NY-1; j++)
62
63
64             for(k=1; k<NZ-1; k++)
65             {
66                 for(l=1; l<NN-1; l++)
67                     if(BB1[i][j][k][l] != BB[i][j][k][l])
68                     {
69                         printf("TEST FAIL!!\n");

```

```

70                                     exit(-1);
71                                     }
72                                 }
73     }
74
75     printf("TEST OK!\n");
76     FILE* fp = fopen("TEST_RIGHT", "w");
77     fclose(fp);
78
79 }

```

图 3-18 编译指示 data 上 block 子句的程序示例

3.7.进入非结构数据区 enter data

enter data 指示表示进入非结构数据区，用于在局存上申请空间、拷贝所需的数据到局存等。

1) 语法

C/C++	#pragma acc enter data [子句列表]
C/C++	
Fortran	!\$ACC ENTER DATA [子句列表]
Fortran	

子句:

async [(整数表达式)];
wait[(整数表达式列表)];
local(变量列表);
copyin(变量列表);
if(条件表达式);
swapahead(数组名((dimension order:,,,....)));
swapalloc(数组名((dimension order:,,,....)));
annotate (暗示子句)。

2) 说明

当主进程执行到 enter data 指示并且下列情况为真时，就会创建非结构数据区，并根据具体的子句进行相应的数据准备。

a)没有 if 子句的出现;

b) if 中条件表达式值非 0(C/C++)或为.true.(Fortran)。

如果有 async 子句，则主进程不需要等待相应的数据准备完成，而直接执行后续代码，可以通过 wait 指示在后续代码适当的位置等待数据准备完成。

非结构数据区最主要的作用是在多个加速计算区之间共享数据，减少数据拷贝，可以将多个加速计算区中都会使用的变量通过非结构数据区，在计算开始之前提前放入局存，这样的好处有两个方面，一是可以在多个加速计算区之间共享数据，二是可以通过异步执行的方式，将主核的执行和数据的准备并行，隐藏部分数据准备的开销。加速计算区可以通过重用数据的方式(present)来使用非结构数据区中的数据。

需要说明的是，非结构数据区中拷贝到局存的数据为每个加速线程私有。由于高速

局存空间较为有限，通常通过非结构数据区拷贝到局存的数据不易过多和过大，应结合具体情况进行控制，非结构数据区通常只存放只读数据。

进入非结构数据区时，进行的是数据准备工作，一般包括申请空间(local)、拷入数据(copyin)、数组的提前转置 (swapahead)、仅申请数组转置后的主存空间 (swapalloc)。

跟 data 的区别在于：data 所定义的数据区是一种结构化数据区，即在代码形式上，data 要求其作用的代码是结构化的代码块，且 data 必须在 parallel 并行区范围内使用。非结构数据区更加灵活，可以包含多个代码段或函数。enter data 指示与 exit data 指示配对使用，分别用于指示进入和退出非结构数据区的位置。enter data 和 exit data 是在 parallel 作用范围之外使用，即在主核代码中使用。

3.7.1 async 子句

参见 3.17.5 [P63]。

3.7.2 wait 子句

参见 3.17.6 [P65]。

3.7.3 local 子句

参见 3.17.2 [P56]。

3.7.4 copyin 子句

参见 3.17.4 [P59]。

3.7.5 if 子句

if 子句的作用和用法与 parallel 上相同，参见 3.4.1 [P19]。

3.7.6 swapahead 子句

对于访问方式不连续的数组访问，其访存性能较低，转置是改善其访存性能的有效方式。swapahead 子句用于实现数组转置。

1) 语法

```
swapahead(数组名((dimension order:,,,.....),.....))
```

2) 说明

与 swap/swapin/swapout 的区别在于，swapahead 创建转置后的数组副本，并完成转置，但是并不将数据拷贝到局存。当位于非结构数据区内的加速计算区需要使用转置后的数组时，需要配合 swaped 暗示子句和 copy/copyin/copyout 子句完成数据到局存的拷贝。

注意：转置功能需要在主存中申请存放转置后数据的空间，如内存空间无法满足需求请谨慎使用。

维度次序 (dimension order), 标识数组维度的编号, 从低维到高维递增, 与 swap/swapin/swapout 一致:

- a) Fortran 程序的数组维度按从左到右编号: 1, 2, 3, 4, ... 其中 1 表示最低维; 描述数组维度顺序也是从左到右排列, 即从低维到高维, 如三维数组的维度顺序是(1,2,3);
- b) C 程序的数组维度按从右到左编号: 1, 2, 3, 4, ... 描述数组维度次序时按照从右到左排列, 即从低维到高维, 如三位数组的维度顺序是(3,2,1)。

3.7.7 swapalloc 子句

对于访问方式不连续的数组访问, 其访存性能较低, 转置是改善其访存性能的有效方式。swapalloc 子句用于在主存申请转置后的数组空间, 并不进行转置, 通常用于在数据区中只被修改的数组。具体用法与 swapahead 子句类似。

3.7.8 暗示子句 annotate

参见 3.17.10 [P68]。

3.8 退出非结构数据区 exit data

exit data 指示表示退出非结构数据区, 用于释放空间, 数据更新等,。

1) 语法

	C/C++
#pragma acc exit data [子句列表]	
	C/C++
	Fortran
!\$ACC EXIT DATA [子句列表]	
	Fortran

子句:

```

async [(整数表达式)];
wait [(整数表达式列表)];
copyout(变量列表);
if(条件表达式);
swapback(数组名((dimension order:,,,...)));
annotate (暗示子句)。

```

2) 说明

当主进程执行到 exit data 指示并且下列情况为真时, 就会销毁非结构数据区, 释放局存中对应的空间, 并根据具体的子句进行相应的数据处理。

- a) 没有 if 子句的出现;
- b) if 中条件表达式值非 0(C/C++)或为.true.(Fortran)。

如果有 async 子句, 则主进程不需要等待相应的数据准备完成, 而直接执行后续代码, 可以通过 wait 指示在后续代码适当的位置等待数据准备完成。

3.8.1 async 子句

参见 3.17.5 [P63]。

3.8.2 wait 子句

参见 3.17.6 [P65]。

3.8.3 copyout 子句

参见 3.17.4 [P59]。

3.8.4 if 子句

if 子句的作用和用法与 parallel 上相同，参见 3.4.1 [P19]。

3.8.5 swapback 子句

swapback 子句和 swapahead、swapalloc 配合使用，用于将 swapahead 转置后的数组或者 swapalloc 申请的数组中的数据，以转置的形式更新回原数组。

1) 语法

```
swapback(数组名((dimension order:,,,....),....))
```

2) 说明

swapback 子句不是必须的，只有当转置的数组中数据被修改，且后续计算需要使用更新后的数据时才使用。

维度次序（dimension order），标识数组维度的编号，从低维到高维递增，与 swap/swapin/swapout 一致：

- a) Fortran 程序的数组维度按从左到右编号：1，2，3，4，...其中 1 表示最低维；描述数组维度顺序也是从左到右排列，即从低维到高维，如三维数组的维度顺序是(1,2,3)；
- b) C 程序的数组维度按从右到左编号：1，2，3，4，...描述数组维度次序时按照从右到左排列,即从低维到高维，如三位数组的维度顺序是(3,2,1)。

3) 示例

略。

3.8.6 暗示子句 annotate

参见 3.17.10 [P68]。

3.9 数据更新指示 update

update 指示用于在主存和局存之间对变量的数值进行更新，需要配合非结构数据区指示使用。

1) 语法

	C/C++
#pragma acc update [子句列表]	
	C/C++
	Fortran
!\$ACC UPDATE [子句列表]	
	Fortran

子句:

```

async [(整数表达式)];
wait [(整数表达式列表)];
host(变量列表);
device(变量列表);
if(条件表达式);
annotate (暗示子句)。

```

2) 说明

当主进程执行到 **update** 指示并且下列情况为真时, 就会根据对应的子句执行数据更新操作。

- a) 没有 **if** 子句的出现;
- b) **if** 中条件表达式值非 0(C/C++)或为.true.(Fortran)。

update 指示不申请新的主存或局存数据空间, 仅根据具体的子句在主存和局存间进行数据更新。对于出现在 **host** 子句中的变量, 会将其在局存中对应的数据更新到对应的主存空间。对于出现在 **device** 子句中的变量, 会将其在主存中的数据更新到对应的局存空间。

update 指示与非结构数据区配合使用, 需要在非结构数据区的作用范围内的主核代码中使用, **host/device** 子句中的变量需要通过非结构数据区在局存中使用 **local** 子句申请空间或通过 **copyin** 子句创建副本。

async 和 **wait** 子句的使用请参见 3.17.5 [P63]和 3.17.6 [P65]。

3.10 组合指示 parallel loop

1) 语法

	C/C++
#pragma acc parallel loop [子句列表] for 循环	
	C/C++
	Fortran
!\$ACC PARALLEL LOOP [子句列表] do 循环 !\$ACC END PARALLEL LOOP	
	Fortran

2) 说明

组合编译指示是指在一个加速计算区内嵌套一个循环编译指示的简短用法, 其含义与显式的指明一个包含循环编译指示的计算区相同, 组合编译指示支持所有计算区和循环编译指示所支持的子句。该编译指示不能出现在另一个加速计算区和循环映射区里

面。

3) 示例

请参考 3.4.7 中的示例[P22]。

3.11 原子操作指示 **atomic**

1) 语法

C/C++
<code>#pragma acc atomic [子句列表] expression-stmt</code>
C/C++
Fortran
<code>!\$ACC ATOMIC [子句列表] expression-stmt !\$ACC END ATOMIC</code>
Fortran

子句:

`uncache(变量列表)`。

2) 说明

编译指示 **atomic** 指定一个存储位置被原子更新，而不会被多个线程同时写，**atomic** 所保护的存储位置通常是位于共享主存的变量。只有对 **x** 指定的变量 **load** 和 **store** 操作是原子的，**expr** 求值不是原子的。对 **x** 所指定的变量进行读或写的操作中不允许有任何任务调度点。为了防止竞争情况，所有可能被并行更新的位置都必须用 **atomic** 指示来保护。

3) 示例

```

1 /*
2  * FILE:ex_3_11.c
3  */
4
5 #include <stdio.h>
6 #include <stdlib.h>
7 #define M 16
8 #define N 1024
9
10 int work1[N];
11 int work2[N];
12 int indx[N],y[N];
13 int x[M];
14
15 void init()
16 {
17     int i;
18
19     for(i=0;i<M;i++)
20         x[i]=0;
21
22     for(i=0;i<N;i++)
23     {
24         y[i]=0;
25         work1[i]=i;

```

```

26      work2[i]=-i;
27      indx[i]=rand()%M;
28  }
29 }
30
31 main()
32 {
33     int i;
34
35     init();
36
37 #pragma acc parallel loop
38     for (i=0; i<N; i++)
39     {
40 #pragma acc atomic
41         x[indx[i]] += work1[i];
42
43         y[i] += work2[i];
44     }
45
46     return 0;
47 }

```

图 3-19 atomic 指示示例

3.11.1 uncache 子句

参见 3.17.9[P68]。

3.12 等待指示 wait

1) 格式

C/C++
#pragma acc wait [(整数表达式列表)] 子句
C/C++
Fortran
!\$ACC WAIT [(整数表达式列表)] 子句
Fortran

其中子句是：

async [(整数表达式)]

2) 说明

wait 指示用于等待之前的异步操作结束，异步操作包括加速计算区的加载(parallel)和之前的 wait 等操作。

wait 指示如果有参数，则参数必须是之前某操作的 async 子句的参数，可以同时等待多个异步操作结束，用逗号分隔；如果 wait 指示没有参数，则等待之前所有的异步操作结束。需要注意的是 wait 指示要在 parallel 的动态区域之外使用。

3) 示例

参见 3.17.5 [P63]中示例。

3.13 函数指示 routine

1) 格式

C/C++
<code>#pragma acc routine</code>
需要在函数的定义之前使用。
C/C++
Fortran
<code>!\$ACC ROUTINE [子句列表]</code>
需要在被修饰的 subroutine 或 function 的声明区域内使用。
Fortran

其中，子句列表是可选项，可根据实际需要设置子句列表；子句列表中的子句以空格分隔。

子句可为下列之一：

memstack(size=非负整数常数[; name=名字字符串]; list=(变量列表));
reusedm(size=非负整数常数);
uncache(变量列表).

2) 说明

routine 指示用于说明将在加速计算区中被调用的函数，编译器将自动为对应的函数创建可在加速设备上执行的版本。

routine 指示的子句目前只能用于 Fortran 中。

限制：

- a) 在 C/C++ 中，**routine** 所修饰的函数中不支持函数内 **static** 变量的使用；
- b) 在 Fortran 中，**routine** 所修饰的函数中不支持带有 **save** 属性的变量。

3) 示例

C/C++
<pre> 1 /* 2 * FILE:ex_3_13-01.c 3 */ 4 5 #include <stdio.h> 6 #include <stdlib.h> 7 #define N 1024 8 9 #pragma acc routine 10 int add(int a, int b) 11 { 12 return a+b; 13 } 14 15 main() 16 { 17 int i; 18 int a[N],b[N]; 19 20 for(i=0;i<N;i++) 21 { 22 a[i]=i; </pre>

```
23      b[i]=-i;
24  }
25
26 #pragma acc parallel loop
27     for(i=0;i<N;i++)
28         a[i]=add(a[i],b[i]);
29
30
31     for(i=0;i<N;i++)
32         if(a[i]!=0)
33         {
34             printf("test failed!\n");
35             exit(-1);
36         }
37
38     printf("test OK!\n");
39
40
41     return 0;
42 }
```

C/C++

Fortran

```
1 !FILE:ex_3_13-02.f90
2
3 function add (a,b)
4 !$acc routine
5     integer add, a, b
6     add = a + b
7 end function
8
9 program main
10 implicit none
11 integer,parameter::N=1024
12 integer::a(N),b(N)
13 integer::i
14
15 do i=1,N
16     a(i)=i
17     b(i)=-i
18 enddo
19
20 !$ACC PARALLEL LOOP
21 do i=1,N
22     a(i)=add(a(i),b(i))
23 enddo
24 !$ACC END PARALLEL LOOP
25
26 do i=1,N
27     if(a(i) .ne. 0) then
28         print *, "TEST FAILED"
29         stop
30     ENDIF
31 enddo
32
```

```

33 print *, "TEST OK"
34
35 end program

```

Fortran

图 3-20 routine 指示示例

3.13.1 memstack 子句

1) 语法

```
memstack(size=非负整数常数[; name=名字字符串] [; list=(变量列表)])
```

2) 说明

加-b 选项运行众核程序时，从核函数栈放在从核的局存中。如果某个函数的栈的容量超出局存的大小，会造成程序运行出错。**memstack** 子句将本从核函数的栈数组放到主存，以减少栈空间对局存的占用，从而可以让程序在-b 选项下正确运行。

memstack 有三个参数 **size**, **name** 和 **list**，其中 **size** 参数是必需的。

size 指示栈数组占用主存空间的大小。

name 给栈数组占用的主存空间取的名字。

list 是放入主存的栈数组列表，列表中的变量顺序指示了栈数组在主存中的排列顺序。栈数组放在主存中访问速度较慢，为提高程序性能，一般需要在执行过程中使用 **data** 指示逐步拷贝部分数据。通常每个数据需要发起一次拷贝，但如果拷贝的多个数据在主存中是连续的，则可以一次拷贝包含多个数组的大块数据，提高数据拷贝的效率。**list** 参数的作用是按需排列栈数组的在主存的存放顺序，便于编译器优化数据拷贝。

3) 示例

```

1 !FILE:ex_3_13_1.f90
2
3 subroutine sub1()
4   implicit none
5   !$acc routine   memstack(size=40960;name=private_region1; list=(A, C, D, B,
E))
6   integer :: A(2048), B(2048),C(2048), D(2048), E(2048)
7   integer :: i, s1
8
9   !$acc data local(E)
10
11  !$acc data copyin(A, C, D)
12    do i = 1, 2048
13      E(i) = A(i) + C(i)
14      E(i) = E(i) + D(i)
15    end do
16  !$acc end data
17
18  !$acc data copyin(B, D)
19    do i = 1, 2048
20      s1 = E(i) + B(i)
21      s1 = s1 + D(i)
22    end do
23  !$acc end data
24
25  !$acc end data

```

```

26 end subroutine
27
28
29 program main
30     implicit none
31
32 !$acc parallel
33     call sub1()
34 !$acc end parallel
35
36 end program

```

图 3-21 memstack 子句示例

3.13.2 reuseldm 子句

1) 语法

reuseldm([size=非负整数常数], [ucaller])

2) 说明

reuseldm 子句有 size 和 ucaller 两种参数，在使用时必须且只能带一个参数。

若 reuseldm 子句带 size 参数时，指示函数中同一层次的多个 data 语句拷贝的数据复用同一块 LDM 空间。size 参数指示了 LDM 空间的大小。当需拷贝的数据总大小超过 LDM 的可用容量时，可以使用该子句实现 LDM 空间的重用。注意，重用只发生在同一层次的数据语句之间，嵌套的数据语句使用的 LDM 空间不是重用而是叠加。

若 reuseldm 子句带 ucaller 参数时，一般用在多层函数调用中的被调函数中，指示被调函数 (callee) 复用调用函数 (caller) 申请的 LDM 空间。该函数在 caller 中被调用时可用的 LDM 地址，是被调函数(callee)可以使用的 LDM 空间的首地址。

3) 示例

```

1 !FILE:ex_3_13_2.f90
2
3 subroutine sub2()
4     implicit none
5 !$acc routine reuseldm(ucaller) memstack(size=4096)
6     integer :: T(1024)
7     integer :: i
8 !$acc data copyout(T)
9     do i = 1,1024
10         T(i) = i
11     end do
12 !$acc end data
13 end subroutine
14
15 subroutine sub1()
16     implicit none
17 !$acc routine reuseldm(size=32768) memstack(size=49152;
name=private_region1; list=(A, C, D, B, F, E))
18     integer :: A(2048), B(2048), C(2048), D(2048), E(2048), F(2048)
19     integer :: i, s1
20
21 !$acc data local(E) copyin(D) ! 1-1
22

```

```

23 !$acc data copyin(A, C)           ! 2-1
24     do i = 1, 2048
25         E(i) = A(i) + C(i)
26         E(i) = E(i) + D(i)
27     end do
28 !$acc end data
29
30     call sub2()
31
32 !$acc data copyin(B, F)           ! 2-2
33     do i = 1, 2048
34         s1 = E(i) + B(i)
35         s1 = s1 + F(i)
36         s1 = s1 + D(i)
37     end do
38 !$acc end data
39
40 !$acc end data
41 end subroutine
42
43
44 program main
45     implicit none
46
47 !$acc parallel
48     call sub1()
49 !$acc end parallel
50 end program

```

图 3-22 reuseldm 子句示例

示例中，2-1、2-2 的 data 语句数据拷贝会复用同一块 LDM 空间，1-1 和 2-1 的 data 语句使用的 LDM 空间是叠加的。函数 sub2 复用 sub1 申请的 LDM 空间，空间的起始地址跟 2-2 的起始地址相同。

3.13.3 uncache 子句

参见 3.17.9[P68]。

3.14 让出加速器控制权限 schedyield

1) 语法

C/C++	#pragma acc schedyield
Fortran	!\$ACC SCHEDYIELD

2) 说明

schedyield 用于指示加速任务调度器退出，让出对加速核心和相关资源的控制。当

程序中需要启动由非 OpenACC 编写的加速核心(kernel)时,比如需要直接启动 (spawn) 由加速线程库编写的加速核心,在启动(spawn)加速核心之前,需要使用 schedyield 指示控制 OpenACC 任务调度器让出对加速核心的控制,否则可能会出错。使用加速线程库编写的被 OpenACC 加速计算核心调用的子函数,不需要使用该指示。

OpenACC 任务调度器退出后会在遇到下一个加速计算区时自动恢复运行。该指示属于可执行指示,由主线程执行,在加速计算区或数据区的范围内使用运行时可能会发生不可预知的错误。

3.15 探测指示 gapfinder

1) 语法

```
gapfinder
```

2) 说明

在使用分布规约(3.17.8)功能等一些需要所有加速线程协同完成的功能时,有可能发生部分加速线程没有机会去处理相应的请求的情况,从而导致性能下降或阻塞。发生这种情况时,可以在加速区内适当的位置添加探测指示,当加速线程执行到探测指示时,会检测是否有未处理的请求,如果有则进行处理。该指示会有一定的开销,请谨慎使用。

3.16 缓存刷新指示 flush

1) 语法

	C/C++
#pragma acc flush	
	C/C++
	Fortran
!\$ACC FLUSH	
	Fortran

2) 说明

缓存刷新指示在加速计算区代码中使用,功能为将加速线程的数据缓存中的数据更新回主存。通常在提供有硬件 Cache 的众核平台上使用,将加速线程 Cache 中的数据刷新回主存,该指示无参数,执行后 Cache 中所有的数据将更新回主存并将 Cache 状态置为无效。在每个加速计算区结束时默认刷新加速线程的数据 Cache。

3.17 编译指示子句

本章主要介绍 OpenACC*编译指示中的子句。OpenACC*加速编译指示的子句大部分是对加速代码中的数据属性进行说明和限定;大部分子句接受一个变量列表参数,变量列表中的变量用逗号分隔,同时这些变量必须是当前代码结构块可见的。在编译指示子句里的所有变量要求是当前代码结构块可见的,可按照需求重复出现,但同一变量不能在多个子句中同时使用,除非有明确说明。

本章主要介绍可以作为不同编译指示子句出现的子句,对于仅在特定编译指示中使用的子句在对应的编译指示章节介绍。

3.17.1 私有化子句 `private/firstprivate`

3.17.1.1 `private`

1) 语法

```
private(变量列表)
```

2) 说明

`private` 子句的作用是声明变量列表中的变量为每个加速线程私有。

在 `private` 子句中说明一个变量的行为如下：在相关联的代码结构块的执行过程中分配一个有自动存储周期的新对象（在当前代码结构块中是可见的），新对象的大小和对界属性由变量类型决定，且新对象的初始值是不确定的（与下文 `firstprivate` 最大的区别），其结果也不更新至主进程的对应内存中。

`private` 子句限制如下：

- ◆ 出现在 `reduction` 子句的变量必须是加速线程私有的；
- ◆ 变量列表中不支持子数组形式。

C/C++

- ◆ 在 `private` 子句中说明的变量不能有不完全的类型或引用类型。

C/C++

Fortran

- ◆ 出现在 `private` 子句里的变量必须是可定义的，或者是可分配的数组。
- ◆ 出现在 `private` 子句里的可分配数组，必须在结构的出、入口有“非当前分配”的分配状态。
- ◆ 假定大小的数组不能出现在 `private` 子句里。
- ◆ 出现在语句名字表中、变量格式表达式中和用来定义函数的表达式中的变量，不能出现在 `private` 子句里。

Fortran

3) 示例

```
1 /*
2  * FILE:ex_3_16_1_1.c
3  */
4
5 #include<stdio.h>
6 #include<stdlib.h>
7 #define N 100
8 int A[N];
9 main()
10 {
11     int i;
12     #pragma acc parallel private(i)
13     {
14         for( i = 0; i < N; ++i )
15         {
16             A[i] = i;
17         }
18     }
19
20     for( i = 0; i < N; i++ )
21     {
22         if(A[i]!=i)
```

```

23      {
24          printf("test fail!\n");
25          exit(0);
26      }
27  }
28  printf("test ok!\n");
29
30  return 0;
31 }

```

图 3-23 private 子句示例

3.17.1.2 firstprivate

1) 语法

```
firstprivate(变量列表)
```

2) 说明

变量列表中的变量语义与private子句类似，不同的是firstprivate子句需初始化每个新的私有变量，在结构块中有一个隐含的初始化说明，并且其初始化值为该变量在主进程中原始对象的值。

C/C++

对非数组类型的变量，初始化就像赋值；对数组（可以是多维），初始化就像从源数组拷贝元素到新数组的对应元素。

C/C++

Fortran

初始化和赋值一样。

Fortran

firstprivate 子句限制与 private 子句的限制一致。

3) 示例

```

1 /*
2  * FILE:ex_3_16_1_2.c
3  */
4
5 #include<stdio.h>
6 #include <stdlib.h>
7
8 #define N 100
9 int A[N];
10 main()
11 {
12     int i,a=1;
13     #pragma acc parallel firstprivate(a)
14     {
15         #pragma acc loop
16         for( i = 0; i < N; ++i )
17         {
18             A[i] = a+i;
19         }
20     }
21     for( i = 0; i < N; i++ )
22     {
23         if(A[i]!=i+1)

```

```

24      {
25          printf("test fail!\n");
26          exit(0);
27      }
28  }
29  printf("test ok!\n");
30
31  return 0;
32 }

```

图 3-24 private 子句示例

3.17.2 本地化子句 local

1) 语法

```
local(varlist)
```

2) 说明

varlist 是变量列表，变量之间以逗号分割，支持标量、数组、子数组。和数据拷贝子句中变量列表的含义相同。

变量列表中的变量、数组等需要在加速设备内存中申请空间，改变主存中同名变量的值不会影响到加速设备，加速设备上对这些变量值的改变也不影响主存中的变量，local 变量的初始值是不确定的。

与数据拷贝子句 copy/copyin/copyout 的区别是，local 子句没有主存和设备内存之间的数据拷贝。

3.17.3 规约子句 reduction

1) 语法

```
reduction(operator:list)
```

其中，operator 指明操作类型，具体支持的操作在下面说明；list 为变量列表，变量之间以逗号分隔。

2) 说明

规约子句 reduction 可以在加速计算区及并行循环上使用，用来并行完成单个标量型变量上的规约计算。

编译器会在每个线程上为 list 表中的每个变量创建一个私有副本，并根据 operator 及变量的数据类型对该私有副本进行初始化；在指定了 reduction 子句的区域的结尾处，使用 operator 操作符对同一个变量的所有私有副本进行一次并行规约操作，并将规约计算的结果与变量的原始值合并，最终结果写回原始变量中。

operator 支持的操作及规约变量副本的初始值如表 3-3 所示。

表 3-3 operator 支持的操作及规约变量副本的初始值

C/C++		Fortran	
operator	初始值	operator	初始值
+	0	+	0
*	1	*	1
max	极小值	max	极小值
min	极大值	min	极大值

&	~0	iand	所有位为 1
	0	ior	0
^	0	ieor	0
&&	1	.and.	.ture.
	0	.or.	.false.
		.eqv.	.true.
		.neqv.	.false.

规约变量的数据类型必须是对所指定的 operator 有效的，支持基本的数值类型包括 C/C++ 语言的整型、浮点型和复数类型以及 Fortran 语言的 integer、real、double precision、complex、logical 等数据类型。

需要注意的是，在规约计算中值结合的顺序是不确定的，因此，比较串行和并行执行或者比较不同的并行执行（即使线程数目相同），并不能保证得到的结果的每一位完全相同（例如浮点异常）。

3) 示例

图 3-25 所示例子测试在 parallel 编译指示上使用 reduction 子句。

```

1 /*
2  * FILE:ex_3_16_3-01.c
3  */
4
5 #include <stdio.h>
6 #include <stdlib.h>
7
8 #define SIZE    1024
9
10 long A[SIZE];
11
12 main()
13 {
14     int i;
15     long A_total = 123, tmp_a;
16
17 #pragma acc parallel private(tmp_a) copyout(A) reduction(+:A_total)
18     {
19         tmp_a=0;
20 #pragma acc loop
21         for(i = 0; i < SIZE; i++)
22         {
23             A[i]=i;
24             tmp_a += A[i];
25         }
26         A_total += tmp_a;
27     }
28
29     //check
30     if(A_total != (SIZE/2*(SIZE-1)+123))
31     {
32         printf("TEST FAILED, A_total = %d.\n", A_total);
33         exit(-1);
34     }
35     printf("TEST OK.\n");
36

```

```

37     return 0;
38 }

```

图 3-25 reduction 子句的例子 1

图 3-26 所示例子测试在 loop 编译指示上使用 reduction 子句。

```

1 /*
2  * FILE:ex_3_16_3-02.c
3  */
4
5 #include <stdio.h>
6 #include <stdlib.h>
7
8 #define SIZE    1024
9
10 long B[SIZE];
11
12 main()
13 {
14     int i;
15     long B_total = 124;
16
17 #pragma acc parallel copyout(B)
18     {
19 #pragma acc loop reduction(+:B_total)
20         for(i = 0; i < SIZE; i++)
21         {
22             B[i]=i;
23             B_total += B[i];
24         }
25     }
26
27     //check
28     if(B_total != (SIZE/2*(SIZE-1)+124))
29     {
30         printf("TEST FAILED, B_total = %d.\n", B_total);
31         exit(-1);
32     }
33     printf("TEST OK.\n");
34
35     return 0;
36 }

```

图 3-26 Reduction 子句的例子 2

图 3-27 所示例子测试在 parallel 和 loop 编译指示上同时使用 reduction 子句。

```

1 /*
2  * FILE:ex_3_16_3-03.c
3  */
4
5 #include <stdio.h>
6 #include <stdlib.h>
7
8 #define SIZE    1024
9
10 long A[SIZE], B[SIZE];

```

```

11
12 main()
13 {
14     int i;
15     long A_total = 123, B_total = 124, tmp_a;
16
17 #pragma acc parallel private(tmp_a) copyout(A,B) reduction(+:A_total)
18     {
19         tmp_a=0;
20 #pragma acc loop reduction(+:B_total)
21         for(i = 0; i < SIZE; i++)
22         {
23             A[i]=i;
24             B[i]=i;
25             tmp_a += A[i];
26             B_total += B[i];
27         }
28         A_total += tmp_a;
29     }
30
31     //check
32     if(A_total != (SIZE/2*(SIZE-1)+123))
33     {
34         printf("TEST FAILED, A_total = %d.\n", A_total);
35         exit(-1);
36     }
37     if(B_total != (SIZE/2*(SIZE-1)+124))
38     {
39         printf("TEST FAILED, B_total = %d.\n", B_total);
40         exit(-1);
41     }
42     printf("TEST OK.\n");
43
44     return 0;
45 }

```

图 3-27 Reduction 子句的例子 3

3.17.4 数据拷贝子句 **copy/copyin/copyout**

对众核加速编程模型而言，内存一般分为主存（host memory）和设备内存（device memory）两部分。主进程的数据空间在主存分配，加速线程的数据空间则既可以在主存上分配，也可以在设备内存上分配。对于加速线程而言，主存容量大，但访问速度慢；设备内存容量小，但速度快。要保证加速线程高效运行，必须尽量保证其操作数据在设备内存里，因此就需要先把主存里的数据分段拷贝到设备内存，完成计算后再拷贝回主存，为此 OpenACC*提供了数据拷贝的编译指示。

1) 语法

```

copy (varlist)
copyin (varlist)
copyout(varlist)

```

其中，varlist 为变量列表，变量之间以逗号分隔。

子句含义：

- a) copy 子句：varlist 中的变量，在计算前，首先将它们的值从主存拷贝到设备内

存；计算结束后，将新的值从设备内存拷回到主存；通常 `varlist` 中的变量在加速代码段中是先读后修改的；

- b) `copyin` 子句: `varlist` 中的变量，仅仅在计算前将它们的值从主存拷贝到设备内存，在相关的计算结束后，并不把加速设备内存上的内容拷贝回主存；通常 `varlist` 中的变量在加速代码段中是只读的；
- c) `copyout` 子句: `varlist` 中的变量，仅仅在计算结束后将它们的值从设备内存拷回到主存；通常 `varlist` 中的变量在加速代码段中是写覆盖的；
- d) 在加速设备中，由于局存有限，`copy` 指定的数组可能会分块传输。如果数组太大且不能分块，则不能使用 `copy` 系列子句指定，可考虑使用 `cache` 子句。

2) 说明

默认情况下，`copy/copyin/copyout` 会根据循环的划分方式将对应的数据进行分割；如果用户用暗示语句指出数据需要全部拷入，`copyin` 会将对应的数据整个拷贝到加速设备内存上，一般用于较小的只读数据。

`varlist` 是变量列表，变量之间以逗号分割，支持标量、数组、子数组，其中对于子数组的描述规则如下：

C/C++	
<code>arr[kk][*]</code>	//kk 对应的最低维数据
<code>arr[kk][lower:upper:max_length]</code>	//kk 对应的最低维 lower-upper 的数据
Fortran	
<code>arr(*,kk)</code>	! kk 对应的最低维数据
<code>arr(lower:upper:max_length, kk)</code>	! kk 对应的最低维 lower-upper 的数据

其中，`lower` 和 `upper` 表示数组在该维度上的下标区间，`lower` 不能小于数组下标的最小值，`upper` 不能超过数组下标的最大值。`Lower` 和 `upper` 可以是整数表达式，不支持其他的情况。`max_length` 只能是常数，用于描述数组某一维区间跨度的最大值，该参数是可选的。

当数组某一维的区间长度可以确定时，不需要给出 `max_length` 参数。

当数组某一维的区间长度不能确定时，即 `upper-lower+1` 不是常数，通常将动态申请所需要的数据空间，而动态申请释放空间的开销相比较较大，因此为减少程序的性能损失，用户可以用 `max_length` 参数给出该区间跨度的最大值，编译系统将依据 `max_length` 的值分配静态空间。`max_length` 参数最多只能出现在数组的某一维上。

注：当 `copy` 作为 `data` 的子句时，`varlist` 的描述允许使用子数组的形式；当 `copy` 作为 `parallel` 的子句时，`varlist` 仅支持标量和数组，不支持子数组的形式。由于每个加速线程有独立的设备内存区域，因此 `varlist` 中的标量只能拷入设备内存，而不能更新回主存，否则存在多个副本，可能导致结果不一致，编译器会根据这个原则自动忽略对标量的拷出操作。对于 `parallel` 上使用 `copy` 子句描述的数组变量，编译器默认按照分布拷贝进行分析和处理，即数组访问跟后续的循环 `loop` 划分有无关联，如果没有关联，会忽略对数组的拷贝操作，如果要将整个数组拷贝到每个加速线程的局存，需要配合 `entire` 暗示子句使用，一般针对只读的小数组。

3) 示例

```
1 /*
2  * FILE:ex_3_16_4-01.c
3  */
```

```

4
5 #include <stdio.h>
6 #include <stdlib.h>
7
8 #define N 1024
9
10 main()
11 {
12     int i;
13     int A[N],B[N],C[N];
14
15     for(i=0;i<N;i++)
16     {
17         A[i]=i;
18         B[i]=-i;
19     }
20
21 #pragma acc parallel loop copyin(A,B) copyout(C)
22     for (i = 0; i < N; i++)
23     {
24         C[i] = A[i]+B[i];
25     }
26
27     for (i = 0; i < N; i++)
28         if(C[i]!=0)
29         {
30             printf("Test FAILED!\n");
31             exit(-1);
32         }
33
34     printf("Test OK!\n");
35     return 0;
36 }

```

图 3-28 数据拷贝子句例子 1

在图 3-28 中所示例子 1 是 copy 编译指示的典型用法，A、B 数组只读，所以只需拷进，C 数组只写，所以只需拷出。如果数组所需空间超过局存限制，循环迭代将以任务分担的方式由多个加速线程执行，相应的数组 A、B 和 C 也会根据循环的划分方式进行适当的分割。例如执行 i=0 迭代的线程只需要 A[0]、B[0]和 C[0]三个元素的数据。

```

1 /*
2  * FILE:ex_3_16_4-02.c
3  */
4
5 #include <stdio.h>
6 #include <stdlib.h>
7
8 #define N 1024
9
10 main()
11 {
12     int x[N];
13     int i;
14     int xi,yi,zi;
15
16     for(i=0;i<N;i++)

```

```

17      x[i]=i;
18
19 #pragma acc parallel loop copyin(x)
20     for (i = 0; i < N; i++)
21     {
22         xi = x[ i + 0 ];
23         yi = x[ i + 1];
24         zi = x[ i + 2];
25     }
26     return 0;
27 }

```

图 3-29 数据拷贝子句例子 2

在图 3-29 中，在一次循环迭代计算中，需要访问 $x[i]$ 、 $x[i+1]$ 、 $x[i+2]$ 三个元素，在这种情况下，编译器会根据循环的分块情况自动拷贝所需的相关数据到加速设备内存中。

```

1 !FILE:ex_3_16_4-03.f90
2
3 program test
4 implicit none
5 integer::i,K
6 INTEGER,PARAMETER::NUM=128
7 INTEGER,PARAMETER::K_SIZE=128
8 INTEGER::ICCG,indwfd
9 integer(8)::pvkd(K_SIZE,NUM,NUM)
10 integer(8)::cg
11 integer::id2pos1(NUM)
12 integer::id2pos2(NUM)
13
14 do i=1,NUM
15     id2pos1(i)=i
16     id2pos2(i)=NUM-i+1
17 enddo
18
19 !$ACC PARALLEL LOOP
20 do i =1,NUM
21     ICCG = id2pos1(i)
22     indwfd=id2pos2(i)
23 !$ACC DATA COPY(pvkd(*, ICCG, indwfd))
24     do k=1,K_SIZE
25         cg=pvkd(k, ICCG, indwfd)
26     enddo
27 !$ACC END DATA
28 enddo
29 !$ACC END PARALLEL LOOP
30 end program test

```

图 3-30 data copy 指示示例 1

在图 3-30 的示例中，data copy 指示出拷贝数组 pvkd 中最高维为 indwfd、次高维为 ICCG 对应的最低维数组数据到设备内存中，并在 DATA 区域结束时拷回主存。

```

1 !FILE:ex_3_16_4-04.f90
2
3 subroutine func(A_slice, n)
4     integer n

```

```

5   integer A_slice(n)
6   integer i,j,lower,length,upper,block
7   block=128
8   do i=1,n/block
9       lower=(i-1)*block+1
10      length=block
11      upper=i*block
12  !$ACC DATA COPY(A_slice(lower:upper:256))
13      do j=1,length
14          A_slice((i-1)*block+j)=(i-1)*block+j
15      enddo
16  !$ACC END DATA
17  enddo
18 end subroutine
19
20 program main
21     integer A(256,256,256)
22     integer j,k
23  !$ACC PARALLEL LOOP
24      do k=1,100
25          do j=1,100
26              call func(A(1:256, j,k), 256)
27          enddo
28      enddo
29  !$ACC END PARALLEL LOOP
30 end program

```

图 3-31 data copy 指示示例 2

在图 3-31 的示例中，data copy 指示添加在 parallel 内被调用的函数内，用于对函数内用到的大数组进行分块处理。

3.17.5 异步子句 async

1) 语法

```
async [(整数表达式)]
```

2) 说明

async 子句可以作为 parallel、wait 等指示的子句使用，其含义表明相应编译指示的操作将以异步方式执行，即只启动对应的操作，而不用等待操作的完成。通常需要与 wait 指示配合使用，使用 wait 指示等待相应操作的完成。

async 的参数是非负整数标量，或者是 OpenACC*预定义的参数，当参数是非负整数标量时，其含义是异步操作队列的编号，即当前的操作将放入对应编号的异步操作队列中执行，相同编号的不同操作将按先后顺序执行，不同编号队列的执行相互独立。当不指定参数时，使用默认的异步操作队列。

3) 示例

```

1 /*
2  * FILE:ex_3_16_5-01.c
3  */
4
5 #include <stdio.h>

```

```

6
7 #define NUM 1024
8 int a[NUM];
9 int b[NUM];
10 int c[NUM];
11 main()
12 {
13     int i;
14     #pragma acc parallel loop async(0) copyout(a)
15     for(i=0;i<NUM;i++)
16     {
17         a[i]=1;
18     }
19     #pragma acc parallel loop async(1) copyout(b)
20     for(i=0;i<NUM;i++)
21     {
22         b[i]=1;
23     }
24     #pragma acc parallel loop wait(0,1) copyout(c) copyin(a,b) async(2)
25     for(i=0;i<NUM;i++)
26     {
27         c[i]=a[i]+b[i];
28     }
29     #pragma acc wait(2)
30
31     return 0;
32 }

```

C/C++

Fortran

```

1 !FILE:ex_3_16_5-02.f90
2
3 program AA
4     integer NUM
5     parameter(NUM=1024)
6     integer a(NUM)
7     integer b(NUM)
8     integer c(NUM)
9     integer i;
10 !$ACC parallel loop async(0) copyout(a)
11     do i=1,NUM
12         a(i)=1
13     enddo
14 !$ACC end parallel loop
15 !$ACC parallel loop async(1) copyout(b)
16     do i=1,NUM
17         b(i)=1
18     enddo
19 !$ACC end parallel loop
20 !$ACC parallel loop wait(0,1) copyout(c) copyin(a,b) async(2)
21     do i=1,NUM
22         c(i)=a(i)+b(i)
23     enddo
24 !$ACC end parallel loop
25 !$ACC wait(2)

```

```
26 end program
```

```
Fortran
```

图 3-32 async 子句示例

3.17.6 等待子句 wait

1) 语法

```
wait [(整数表达式列表)]
```

2) 说明

wait 子句可以作为 parallel、data 等指示的子句使用，其作用与 wait 指示相同，用于等待之前的异步操作完成。当编译指示中出现 wait 子句时，必须先等待 wait 子句关联的异步操作完成才执行相应指示所对应的操作。wait 子句的参数必须与之前某个 async 子句的参数一致，wait 子句的参数可以为多个整数表达式，使用逗号间隔；当 wait 子句无参数时，等待之前所有的异步操作完成。

3) 示例

```
1 /*
2  * FILE:ex_3_16_6.c
3  */
4
5 #include <stdio.h>
6
7 #define NUM 64
8
9 main()
10 {
11     int i,a[NUM],c[NUM];
12     long result;
13
14     #pragma acc parallel loop async(0) copyout(a)
15     for(i=0;i<NUM;i++)
16         a[i]=1;
17
18     #pragma acc parallel loop wait(0) copyout(c) copyin(a)
19     for(i=0;i<NUM;i++)
20         c[i]=a[i]+2;
21
22     result=0;
23     for(i=0;i<NUM;i++)
24         result+=c[i];
25
26     printf("result=%ld\n", result);
27
28     return 0;
29 }
```

图 3-33 wait 子句示例

3.17.7 数据重用子句 **present**

1) 语法

```
present (变量列表)
```

2) 说明

present 可以作为 **parallel** 和 **data** 的子句使用，用于指定数据已经在之前的操作中被拷贝到设备内存，后面的一段代码将重用之前拷入的数据。变量列表内的变量可以是标量、数组或子数组。

present 主要用在被加速计算区所调用的函数(简称加速函数)中，当数据在加速函数执行期间已经存在于设备内存中，则可以在加速函数内使用 **present** 重用相应的数据。

present 可重用的数据包括主要是非结构数据区上指定的数据。使用 **present** 时，必须保证 **present** 所指明的数据已经存在于设备内存中。数据的一致性由用户程序保证。

3) 示例

```
// FILE: ex_3_16_7.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define M 512
5 #define N 1024
6
7 int aa[N];
8 int bb[M][N], cc[N][N];
9
10 int main()
11 {
12     int c;
13     int i, j;
14     FILE *fp;
15
16     c = 4;
17
18     for(j = 0; j < N; j++)
19         aa[j] = j;
20
21     #pragma acc enter data copyin(aa)
22
23     #pragma acc parallel loop copyout(bb) present(aa)
24     for(i = 0; i < M; i++)
25         for(j = 0; j < N; j++)
26             bb[i][j] = i + aa[j] + c ;
27
28     #pragma acc parallel loop copyout(cc) present(aa)
29     for(i = 0; i < N; i++)
30         for(j = 0; j < N; j++)
31             cc[i][j] = i + aa[j] + c ;
32
33     #pragma acc exit data
34 }
```

图 3-34 **present** 子句的程序示例

3.17.8 分布规约子句 **dis_reduction**

1) 语法

dis_reduction(operator:list)

其中，**operator** 用于指明操作类型，具体支持的操作在下面说明；**list** 为数组变量列表，变量之间以逗号分隔。

2) 说明

分布式规约子句 **dis_reduction** 可以在加速计算区上使用，用于实现对大数组的离散更新，类似 `Arr[idx] += newVal` 这种模式；适用的程序具有以下特点：**1)** 被更新的数组长度较大，单个加速线程无法将其完整存放到本地局存中；如果单个线程能将对象数组放在本地局存，推荐使用普通规约子句；**2)** 每次访问的具体位置是离散的，即 `idx` 的值不是连续线性变化的；**3)** 可能存在不同加速线程更新同一个位置的情况（写冲突）。

使用限制：**1)** `list` 中只支持数组类型变量；**2)** 对象数组的访问方式只能是做累加或累减操作，即以下四种形式：`arr[idx] +=...`、`arr[idx] -=...`、`arr[idx]=arr[idx]+...`、`arr[idx]=arr[idx]-...`，不能出现其他形式的对象数组访问；**3)** 该操作内部需要由加速阵列中所有加速线程共同参与执行；**4)** 对象数组的总长度要小于“加速线程空间的局存空间”乘以“总加速线程数”，原因是对象数组是分布存放于每个加速线程上的，以此解决单个加速线程空间不够的问题；**5)** `list` 中最多同时支持 4 个数组变量，此限制仅针对一个加速计算区。

operator 目前仅支持+和-两种。

分布规约属于全局操作，要求所有加速线程都能适时参与，因此，如果对象数组的更新语句出现在特定的代码分支内，而且这些分支只有部分加速线程能够执行，这种情况下有可能导致分布规约操作阻塞。当使用分布规约后程序运行出现阻塞，可以尝试在代码中所有加速线程都能执行的位置使用 `gapfinder` 指示。

3) 示例

```
// FILE:ex_3_16_8.c
1 #include <stdlib.h>
2 #include <stdio.h>
3 #define nCell 40000
4 #define nFace 112000
5 double x_double[nCell];
6 int face_cel_a[nFace];
7 int main()
8 {
9     int i;
10    int ii;
11    double newValue = 0.079;
12    for (i = 0; i < nCell; i++)
13    {
14        x_double[i] = 0.0;
15    }
16    for (i = 0; i < nFace; i++)
17    {
18        face_cel_a[i] = rand() % nCell;
19    }
20    #pragma acc parallel loop tile(1750) \
21        dis_reduction(+:x_double) copyin(face_cel_a)
22    for (i = 0; i < nFace; i++)
23    {
```

```

24      ii = face_cel_a[i];
25      x_double[ii] +=  newValue;//x_double 要进行更新，可能有写冲突
26  }
27 }

```

3.17.9 不可缓存子句 **uncache**

1) 语法

```
uncache (变量列表)
```

2) 说明

在配置有硬件 Cache 的众核平台中，加速计算内的访存操作默认会通过硬件 Cache 进行优化。但是在某些情况下，使用 Cache 可能会带来性能的损失或者正确性的问题，因此提供不可缓冲子句，将指定的变量描述为不可 Cache 属性，通知编译器尽量不适用 Cache 优化加速计算区内的访存操作。

该子句可以作为 **parallel**、**routine**、**atomic** 的子句出现，且只在该指示的作用范围内有效。若在从核函数的变量在 **routine** 指示中设置了该属性，则这些变量在函数的每次调用时都是不可 cache 访问的。

使用限制：1) 对于设置了不可 Cache 属性的变量，在该属性有效范围内，用户不能通过变量名以外的方式(比如指针)访问，否则可能会导致程序正确性问题。2) 在加速段中，若将全局变量标记为不可 Cache 属性，除在 **parallel** 上使用 **uncache** 子句进行描述外，还需在该加速段调用的函数内通过 **routine** 上的 **uncache** 子句进行描述，以保证变量的单一属性。

3.17.10 暗示子句 **annotate**

暗示子句的语法格式如下：

```
annotate ([暗示信息]; 暗示信息)
```

其中暗示信息可以是下列之一：

```

tilemask(array1,array2,...)/ tilemask(dim1:array1,dim2:array2,...);
entire(array1, array2, array3,...);
readonly(varlist);
dimension(array1name(dim1,dim2,...),array2name(dim1,dim2,...),...);
co_compute;
margin_compute;
slice(array1name(access_mode_info), ...);
swaped(数组名((dimension order:,,,....),....));
group_shared(变量列表);
spm_config(group_shared_mode:共享范围:共享空间大小 cache_mode:cache
大小);
profiling/profiling:prof_mode_info,...;

```

暗示子句用于对 OpenACC*其他指示或子句进行辅助说明，暗示子句能否发生作用由编译器根据实际情况进行分析和判断决定。

3.17.10.1 **tilemask**

1) 语法

```

annotate (tilemask(array1,array2,...))
annotate (tilemask(循环迭代变量 i:数组 1, 循环迭代变量 j:数组 2,...))

```

2) 说明

屏蔽循环分割对指定数组的影响，使相关数组相关维的数据不被分割，提升数据传输效率。`tilemask` 暗示子句有两种用法：

用法 1: `tilemask` 暗示子句的参数是数组名列表，一般用于有循环分割的 `loop` 指示。在程序中，一个循环往往是对数组的某一维度进行迭代，数组的这一维是该循环的对应维度。`tilemask` 用于指出列表中的数组不受该循环分块的影响，对应维度的数据能够在该循环分割之前全部拷入设备存储器，从而能够在对应的循环外拷贝相关数据，减少 DMA 次数。

用法 2: `tilemask(循环迭代变量 i:数组 1, 循环迭代变量 j:数组 2,...)`，作用于 `loop` 指示内直接嵌套的所有循环，通过循环迭代变量的方式指定受影响的循环，并屏蔽对应循环的分割对相关数组的影响，这种用法通常与 `tile(循环迭代变量 i:块大小 1,循环迭代变量 j:块大小 2, ...)` 配合使用。

3) 示例

```

1 /*
2  * FILE:ex_3_16_9_1-01.c
3  */
4
5 #include<stdio.h>
6 #include<stdlib.h>
7 int A[64][128], B[64][128], C[64][128];
8
9 int main()
10 {
11     int i, j;
12     for(i=0;i<64;i++)
13         for(j=0;j<128;j++)
14         {
15             A[i][j]=i+j;
16             B[i][j]=i*j;
17         }
18     #pragma acc parallel copyin(A, B) copyout(C)
19     #pragma acc loop
20     for(i = 0; i < 64; i++)
21     {
22         #pragma acc loop tile(16) annotate(tilemask(A, B))
23         for(j = 0; j < 128; j++)
24         {
25             C[i][j] = A[i][j] + B[i][j];
26         }
27     }
28     for(i=0;i<64;i++)
29         for(j=0;j<128;j++)
30         {
31             if(C[i][j]!=(i+j+i*j))
32             {
33                 printf("Test Fail!\n");
34                 exit(0);
35             }
36         }
37     printf("Test OK!\n");

```

```

38     return 0;
39 }

```

图 3-35 annotate 子句示例-tilemask

tilemask 暗示子句用法 1 的示例如图 3-35 所示，tilemask 指示用在内层 loop 指示上，参数为数组 A，B。虽然内层循环的分块大小为 16，但数组的 A，B 的最低维不受此分块影响，最低维的数据将在内层循环之外全部拷入设备内存。而数组 C 的最低维则被相应的划为大小为 16 的块，每次拷入 1 块，分多次拷入。

```

1 /*
2  * FILE:ex_3_16_9_1-02.c
3  */
4
5 #include<stdio.h>
6 #include<stdlib.h>
7 int A[64][128], B[64][128], C[64][128];
8
9 int main()
10 {
11     int i, j;
12     for(i=0;i<64;i++)
13         for(j=0;j<128;j++)
14         {
15             A[i][j]=i+j;
16             B[i][j]=i*j;
17         }
18     #pragma acc parallel copyin(A, B) copyout(C)
19     #pragma acc loop tile(i:1, j:16) annotate(tilemask(j:A, j:B))
20     for(i = 0; i < 64; i++)
21     {
22         for(j = 0; j < 128; j++)
23         {
24             C[i][j] = A[i][j] + B[i][j];
25         }
26     }
27     for(i=0;i<64;i++)
28         for(j=0;j<128;j++)
29         {
30             if(C[i][j]!=(i+j+i*j))
31             {
32                 printf("Test Fail!\n");
33                 exit(0);
34             }
35         }
36     printf("Test OK!\n");
37     return 0;
38 }

```

图 3-36 annotate 子句示例-tilemask

tilemask 暗示子句用法 2 的示例如图 3-36 所示，其效果与图 3-35 的程序相同。

3.17.10.2 entire

1) 语法

```

annotate (entire(array1, array2, array3,...))

```

2) 说明

`entire` 暗示的参数是数组名列表，一般用于 `parallel` 指示的子句，指出列表中的数组要全部拷入每个线程对应的设备内存。

3) 示例

```

1 /*
2  * FILE:ex_3_16_9_2.c
3  */
4
5 #include <stdio.h>
6
7 int A[64], B[128][64];
8
9 int main()
10 {
11     int i, j;
12
13     #pragma acc parallel copyin(A) copyout(B) annotate(entire(A))
14     {
15         #pragma acc loop
16         for(i = 0; i < 128; i++)
17         {
18             for(j = 0; j < 64; j++)
19                 B[i][j] = A[j] + i;
20         }
21     }
22     return 0;
23 }

```

图 3-37 entire 示例

在上图的示例中，数组 A 的全部数据在每个线程对应的设备存储器中都有一份完整的拷贝，数组 B 则是根据并行循环划分的方式，分块分布存储在每个线程的设备存储器中。

3.17.10.3 readonly

1) 语法

```
annotate (readonly(varlist))
```

2) 说明

`readonly` 的参数是变量名列表，一般用于 `parallel` 指示的子句，指出加速计算区范围内的只读变量信息。

3) 示例

```

1 /*
2  * FILE:ex_3_16_9_3.c
3  */
4
5 #include <stdio.h>
6 #include <stdlib.h>
7
8 int A[64][128];
9
10 int main()
11 {

```

```

12     int i, x;
13
14     x = rand()%64;
15     #pragma acc parallel loop copyout(A) annotate(readonly(x))
16     for(i = 0; i < 128; i++)
17         A[x][i] = i;
18     return 0;
19 }

```

图 3-38 readonly 示例

在上图的示例中，readonly 暗示用于辅助说明数组下标访问。在加速计算区内，数组 A 的最高维下标访问 x 为只读变量，编译器在设备存储器上为数组 A 申请空间时，最高维的大小为 1。否则，设备存储器上数组 A 的最高维大小为 64。

3.17.10.4 dimension

1) 语法

```
annotate (dimension(array1name(dim1,dim2,...),array2name(dim1,dim2,...),...))
```

2) 说明

当指针(或动态数组)在加速计算区中以数组的形式使用时，编译器往往无法判断出该指针(或动态数组)所指向数组的维度信息，进而无法在设备存储器中申请适当大小的空间并拷贝数据。dimension 暗示一般用于 parallel 指示的子句，指出指针(或动态数组)所指向数组空间的维度信息，从而帮助编译器在加速计算区中申请设备空间并并拷贝数据。

dimension 暗示中所指出的数组维度信息，Fortran 数组的维度从低维到高维说明，C 数组的维度从高维到低维说明，中间用逗号隔开。

3) 示例

```

1 /*
2  * FILE:ex_3_16_9_4-01.c
3  */
4
5 #include <stdio.h>
6 #include <malloc.h>
7
8 int A[64][128];
9 int **B;
10
11 int main()
12 {
13     int i, j;
14     int *p1, **p2;
15
16     p1 = (int *)malloc(sizeof(A));
17     p2 = (int **)malloc(64*sizeof(int *));
18     for(i = 0; i < 64; i++)
19     {
20         *(p2+i) = (int *)p1+i*128;
21     }
22     B = p2;
23
24     for(i = 0; i < 64; i++)
25         for(j = 0; j < 128; j++)

```

```

26          A[i][j] = i+j;
27
28 #pragma acc parallel copyin(A) copyout(B) annotate(dimension(B(64, 128)))
29 {
30     #pragma acc loop
31     for(i = 0; i < 64; i++)
32         for(j = 0; j < 128; j++)
33             B[i][j] = A[i][j];
34 }
35 return 0;
36 }

```

图 3-39 dimension 示例 1

在上图的示例中，dimension 暗示描述了指针 B 所指向空间的维度信息。

```

1 !FILE:ex_3_16_9_4-02.f90
2
3 program main
4     implicit none
5     integer :: i, j
6     integer , allocatable :: b(:, :)
7     integer :: a(128, 64)
8
9     allocate(b(128, 64))
10
11     do i = 1, 64
12         do j = 1, 128
13             a(j, i) = i+j
14         end do
15     end do
16
17     !$ACC      PARALLEL      COPYIN(a)      COPYOUT(b)
18     ANNOTATE(DIMENSION(b(128, 64)))
19     !$ACC LOOP
20         do i = 1, 64
21             do j = 1, 128
22                 b(j, i) = a(j, i)
23             end do
24         end do
25     !$ACC END LOOP
26     !$ACC END PARALLEL
27 end program

```

图 3-40 dimension 示例 2

在上图示例中，dimension 暗示描述动态数组 b 的维度信息。

3.17.10.5 slice

1) 语法

```
annotate (slice(array1name(access_mode_info), ...))
```

2) 说明

slice 暗示的参数列表是数组名及其访问模式的描述，一般用于 parallel 编译指示的子句，是对 parallel 上数据 copy 子句的补充。slice 暗示用于说明需要拷贝的数组在加速计算区中的访问模式。特别地，如果数组是分段的区间访问时，可以将数组每段的访问

模式都列在参数列表中，并与 `parallel` 指示的 `copy` 子句配合使用，来拷贝数组一个片段或多个片段的数据，提升数据传输的效率和降低对高速局存的占用，其数组访问模式的格式描述如下：

C/C++	
<code>arr[kk][*]</code>	//kk 对应的最低维数据
<code>arr[kk][lower:upper]</code>	//kk 对应的最低维 lower-upper 的数据
Fortran	
<code>arr(*,kk)</code>	! kk 对应的最低维数据
<code>arr(lower:upper, kk)</code>	! kk 对应的最低维 lower-upper 的数据

可以看出 `slice` 的格式与 `copy` 子句中的子数组描述类似，但是二者的含义和用法有所区别。`slice` 是用于描述加速计算区内数组的访问模式，而没有数据拷贝的动作。`*`、`kk`、`lower:upper` 属于 `access_mode_info` 的三种描述方式，`*` 表示该维所有数据均需要访问，`kk`、`lower`、`upper` 中可以是常数、变量或变量表达式，通常情况下变量或变量表达式应在程序中的对应数组访问中实际使用，变量表达式的格式支持 `i`、`i+n`、`i-n`，`x`，其中 `n` 为常数，`x` 为整数变量。

使用时需要注意的包括：

- 当同一个数组出现多种访问模式时，**需要保证每种访问模式是静态可区分的**，例如，不能出现 `A[i-1:i+1][*]`，`A[20:30][*]`，因为无法区分程序中的 `A[i][j]` 属于哪一种模式，这种情况可能会导致无法预知的错误。
- 当多种访问模式之间有数据重叠时，**需要用户保证程序运行过程中的数据一致性**，这种情况下通常用于只读数据。
- `slice` 用于指导编译系统按照访问模式将数据切分成小的数据片段，通常在希望压缩高速局存使用量时采用。需要注意的是当大块数据切分成多个数据片段时，需要多次相对小数据量的数据传输，其数据传输开销比整块数据传输要高。因此在实际使用时需要在局存的空间占用量和数据传输开销之间权衡，在局存空间允许的情况下，尽量使用连续的整块数据传输。

3) 示例

```

1 /*
2  * FILE:ex_3_16_9_5.c
3  */
4
5 #include <stdio.h>
6 #include <stdlib.h>
7
8 int A[100][100][32], B[100][100][32];
9 int tmp1=0, tmp2=0;
10
11 int main()
12 {
13     int i,j,k;
14     #pragma acc parallel loop tile(1,1) copyout(B) copyin(A)
15     annotate(slice(A[i-5:i+5][j][*], A[i][j-5:j+5][*]))
16     for(i=5;i<95;i++)
17         for(j=5;j<95;j++)
18             for(k=0;k<32;k++)
19                 {

```

```

19         tmp1=tmp1 + A[i][j][k] +
20         A[i][j-1][k]+ A[i][j+1][k] +
21         A[i][j-2][k]+ A[i][j+2][k] +
22         A[i][j-3][k]+ A[i][j+3][k] +
23         A[i][j-4][k]+ A[i][j+4][k] +
24         A[i][j-5][k]+ A[i][j+5][k] ;
25
26         tmp2=tmp2 + A[i][j][k] +
27         A[i-1][j][k]+ A[i+1][j][k] +
28         A[i-2][j][k]+ A[i+2][j][k] +
29         A[i-3][j][k]+ A[i+3][j][k] +
30         A[i-4][j][k]+ A[i+4][j][k] +
31         A[i-5][j][k]+ A[i+5][j][k] ;
32
33         B[i][j][k]=tmp1+tmp2;
34     }
35     return 0;
36 }

```

图 3-41 slice 暗示示例

图 3-41 的例子中 A 数组的访问模式较为复杂，当 i, j 固定时(k 维不划分)，所需要访问的数据区间为 $A[i-5:i+5][j-5:j+5][*]$ ，则数据量为 $11*11*32*\text{sizeof}(\text{int})=15488\text{B}=15\text{KB}$ (32 是最低维的长度)，当设备内存紧张时，这么多数据可能无法全部存放，但是通过分析程序可以发现，程序中实际用到的数据并没有这么多，可以进一步分拆成 $A[i-5:i+5][j][*]$ ， $A[i][j-5:j+5][*]$ ，如果按这两种模式来细分所需访问的数据，则只需要 $11*1*32*\text{sizeof}(\text{int}) + 1*11*32*\text{sizeof}(\text{int})=2816\text{B}$ ，会大大减少高速局存占用量。

3.17.10.6 swaped

1) 语法

```
annotate (swaped(数组名((dimension order:,,,...),.....))
```

2) 说明

该暗示子句与 parallel 指示上的 copy 配合使用，用于说明对应的数组存在已经转置的副本，parallel 中将使用已转置后数组中的数据。通常用法是在进入非结构数据区 enter data 上进行数组转置，通过 swaped 暗示和 copy 子句完成数据拷贝。

3) 示例

略。

3.17.10.7 group_shared

1) 语法

```
annotate (group_shared(变量列表))
```

2) 说明

该暗示子句通常与 enter data 上的 copyin 配合使用，用于说明对应的变量存放于共享局存空间，对应的数据副本将由一定数量的加速线程共享使用，通常为有一定空间需求的只读数组。使用该功能需要在 bsub 提交作业时通过 -ldm_share_mode、-ldm_share_size 两个选项进行设定，指定共享的模式和共享空间大小。

-ldm_share_size 用于指定每个加速线程上用于共享的本地 LDM 空间大小，有效的取值范围是 4、8、16、32、64、128，分别表示每个核拿出 4KB、8KB、16KB、32KB、64KB、128KB 的 LDM 空间共享，需要注意的是用于共享的 LDM 空间是从私有 LDM

空间中划出的，用于共享的空间越多，可用的私有 LDM 空间越小。

-ldm_share_mode 用于指定共享的范围和模式，有效的取值包括 0、1、2、4、5。其中 0，表示共享范围是 4 个核，所有从核分组共享，组大小是 4，同一组内加速线程共享同一块 LDM 共享空间，不同组的加速线程共享不同的空间；1 表示共享组大小是 8，2 表示共享组大小是 16，4 和 5 均表示共享组大小是 64，二者略有区别，一般情况下影响可以忽略。

3) 示例

```

/* FILE:ex_3_16_9_7.c */
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 #define N 65536
5
6 int AA[N]; //256KB
7
8 int main()
9 {
10     int i, j;
11     long sum1, sum2;
12     FILE *fp;
13
14     sum1 = sum2 = 0;
15
16     for(j = 0; j < N; j++)
17         AA[j] = j+1;
18
19     #pragma acc enter data copyin(AA) annotate(group_shared(AA))
20
21     #pragma acc parallel loop tile(64) present(AA) reduction(+:sum1)
22     for(i = 0; i < N; i++)
23     {
24         sum1 += AA[i];
25     }
26}

```

图 3-42 group_shared 暗示示例

3.17.10.8 spm_config

1) 语法

```

annotate(spm_config([cache_mode : cache 大小] [group_shared_mode : 共享范围 : 共享空间大小]))

```

2) 说明

该暗示字句可以作为 enter data 的暗示子句出现，用于更改默认的 SPM 空间配置，出现在其他指示中无效。

默认的 SPM 配置由基础编译器或者运行时参数设定，通常会配置 32KB SPM 空间作为加速线程私有数据 Cache 使用，其余作为加速线程私有数据空间使用，没有加速线程共享的 SPM 空间。

可根据课题需要改变默认的 SPM 配置，其中 cache_mode 用于改变默认的 Cache 空间大小，‘cache 大小’的有效值包括：0、32、128，共三种，单位为 KB。

默认没有配置加速线程共享的 SPM 空间，可根据课题需要改变默认的 SPM 配置。group_shared_mode 的 ‘共享范围’ 的有效输入为：64，表明共享空间由 64 个加速线程共享同一段空间；‘共享空间大小’ 的有效输入包括：4、8、16、32、64、128，共五种，

单位为 KB，表明每个加速线程所提供的用于共享的空间大小，总的共享空间大小可由共享范围指定的加速线程数乘上每个加速线程提供的共享空间大小确定。指定数据存放不到 SPM 共享空间需要使用 `group_shared` 暗示字句。

Cache 空间大小的设置后只有等到下次改变设置才会改变，没有更改 cache 大小的加速段将使用最近设置的 Cache 大小。

SPM 共享空间在与设置共享模式的 `enter data` 配对的 `exit data` 时被关闭，在此 `enter data/exit data` 动态执行范围内，共享模式及 cache 大小不能再改变，否则运行时会发生不可预知的错误。

具体的配置根据不同的硬件平台有所差异。另外，改变 SPM 配置有开销，请根据需要谨慎使用。

该功能暂未提供。

3) 示例

略。

3.17.10.9 co_compute

1) 语法

```
annotate (co_compute)
```

2) 说明

`co_compute` 暗示一般用于 `loop` 指示的子句，指定主线程按照一个加速线程的模式来进行计算，主线程和加速线程一起参与并行循环迭代任务的分担。

3) 示例

```
1 /*
2  * FILE:ex_3_16_9_9.c
3  */
4
5 #include <stdio.h>
6 #include <stdlib.h>
7
8 int A[130];
9
10 int main()
11 {
12     int i;
13
14     #pragma acc parallel copyout(A)
15     {
16         #pragma acc loop annotate(co_compute)
17         for(i = 0; i < 130; i++)
18         {
19             A[i] = i;
20         }
21     }
22     return 0;
23 }
```

图 3-43 co_compute 示例

在上图的示例中，并行循环由主线程和加速线程共同分担。若有 64 个加速线程，则主线程承担 $i=65$, $i=129$ 次迭代。

3.17.10.10 margin_compute

1) 语法

```
annotate (margin_compute)
```

2) 说明

margin_compute 暗示一般用于 loop 指示的子句。并行循环的迭代次数被拆分为两部分，一部分是能够被加速线程数整除的规整部分，另一部分是被整除后的余量部分。margin_compute 暗示指定规整部分由加速线程执行，余量部分由主线程执行。

3) 示例

```

1 /*
2  * FILE:ex_3_16_9_10.c
3  */
4
5 #include <stdio.h>
6 #include <stdlib.h>
7
8 int A[132];
9
10 int main()
11 {
12     int i;
13
14     #pragma acc parallel copyout(A)
15     {
16         #pragma acc loop annotate(margin_compute)
17         for(i = 0; i < 132; i++)
18         {
19             A[i] = i;
20         }
21     }
22     return 0;
23 }
```

图 3-44 margin_compute 示例

在上图的示例中，第 0~127 次迭代由加速线程执行，第 128~131 次迭代由主线程执行。

3.17.10.11 profiling

1) 语法

```
annotate(profiling)
annotate(profiling:prof_mode_info,...)
```

2) 说明

profiling 为暗示子句，其中 prof_mode_info 为性能监测模式，现支持 gld,dma, miss_ratio,only_kernel,timer 以上几种模式，多种模式间以逗号间隔。其中 gld 模式与 miss_ratio 模式互斥，不能同时出现在同一核心段中。

profiling 暗示一般用于 parallel 编译指示中，指出该核心加速段需要进行性能统计和分析。若不设置性能监测模式，则默认为 gld,dma 模式性能监测。

需要注意的是，在从核调用函数时会引入访主存操作，且在统计 fortran 核心函数子函数的性能监测数据时也会引入额外的访主存操作，影响统计数据的准确性，为此增加 only_kernel 模式，只统计从核加速主函数的监测数据，而不再统计其子函数监测数据，消除统计误差。

在实际程序的测试过程中可能出现仅针对于整个核心段的性能统计需求。现添加 **timer** 模式，仅统计核心段总节拍数及运行时间。

注：该功能暂无法支持代码段级并行核心段的的同时的性能监测统计。

3) 示例

```

1 /*
2  * FILE:ex_3_16_9_11-01.c
3  */
4
5 #include <stdio.h>
6 #include <stdlib.h>
7
8 void func(int i,int *A)
9 {
10     int c;
11     c = i;
12     A[i] = c;
13 }
14
15 int main()
16 {
17     int i,A[64];
18
19     #pragma acc parallel loop annotate(profiling)
20     for (i=0; i<64; i++)
21         func(i,A);
22
23     return 0;
24 }

```

图 3-44 profiling 示例（1）

在上例中，在核心段中添加 **profiling** 暗示字子句后，编译器会在该核心段前后添加性能检测及统计接口，函数核心段及子函数相关性能检测结果和分析在程序运行后会打印到 **pref_info.log** 文件中，以供参考。

在实际应用过程中，可能会出现程序子函数和主函数并不同时出现在同一文件中的情况，如下例所示：

```

1 /*
2  * FILE: ex_3_16_9_11-02.c
3  */
4
5 #include <stdio.h>
6 #include <stdlib.h>
7
8 #pragma acc routine
8 void func(int i,int *A)
9 {
10     int c;
11     c = i;
12     A[i] = c;
13 }

1 /*
2  * FILE: ex_3_16_9_11-03.c
3  */

```

```

4
5 #include <stdio.h>
6 #include <stdlib.h>
7
8 int main()
9 {
10     int i,A[64];
11
12     #pragma acc parallel loop annotate(profiling)
13     for (i=0; i<64; i++)
14         func(i,A);
15
16     return 0;
17 }

```

图 3-45 profiling 示例 (2)

此时在编译子函数所在文件 `ex_3_16_9_11_02.c` 时，需要添加编译选项 `-profiling`，完成在子函数前后的性能检测接口插装。

在实际应用中，需要对并行程序进行性能监测统计，现功能支持对 MPI 程序的性能监测统计，可通过添加编译选项 `-profiling_mpirank`，在其后添加相关数字以确定性能检测的进程号，默认为 1 号进程，该功能现仅支持 MPI 程序中一个进程的性能监测。

3.17.11 组合子句

1) 语法

```

private(varlist) copyin(varlist)
private(varlist) copyout(varlist)
private(varlist) copy(varlist)
private(varlist) cache(varlist)

```

2) 说明

该子句为 `private` 和 `copy` 子句的组合，首先对 `varlist` 中指定的变量先进行私有化处理（放在主存的私有空间），再进行 `copyin`、`copyout`、`copy` 或 `cache` 操作。

其中 `private` 子句可以换成 `firstprivate` 子句，在这种情况下，`varlist` 中指定的变量先进行私有化处理，并用原始数据初始化每个新的私有化变量。

3) 示例

```

1 /*
2  * FILE:ex_3_16_10-01.c
3  */
4
5 #include <stdio.h>
6 #include <stdlib.h>
7
8 int A[8192];
9
10 int main()
11 {
12     int i, k;
13
14     #pragma acc parallel private(A) copy(A)
15     {
16         #pragma acc loop
17         for(k = 0; k < 64; k++)

```

```

18      {
19 #pragma acc loop tile(1024)
20         for(i = 0; i < 8192; i++)
21         {
22             A[i] = i*10;
23         }
24 #pragma acc loop tile(1024)
25         for(i = 0; i < 8192; i++)
26         {
27             A[i] = A[i] + k;
28         }
29     }
30 }
31 return 0;
32 }

```

图 3-46 private 和 copy 组合子句示例

在上图的示例中，OpenACC*编译器对循环变量 k 进行并行划分。在内层循环中，每个加速线程都需要读写数组 A ，然而数组 A 的大小无法全部拷入设备内存。因此，先把数组 A 私有化，再对内层循环进行分块。每个加速线程从自己的私有主存区每次拷贝数组 A 的一块到设备内存。

```

1 !FILE:ex_3_16_10-02.90
2
3 program main
4     implicit none
5     integer :: i, k, m
6     integer :: a(8192)
7     common a
8
9     do i = 1, 8192
10         a(i) = i
11     end do
12
13 !$ACC PARALLEL LOOP FIRSTPRIVATE(a) CACHE(a:8)
14     do k = 1, 64
15         do i = 1, 8192
16             m = mod(2*i+k, 8192)
17             a(m) = a(m)+k+i
18         end do
19     end do
20 !$ACC END PARALLEL LOOP
21 end program

```

图 3-47 firstprivate 和 cache 组合子句示例

在上图的示例中，每个加速线程都要读写数组 A 的全部数据，而数组 A 不能全部拷入设备内存。程序中对数组 A 的访问是离散的，所以将 A 私有化，并采用 `cache` 的方式提高数组 A 的访问效率。

4 运行时库接口

OpenACC*语言环境支持 OpenACC2.0 中定义的所有运行时库接口，但是由于不恰当地

使用运行时库接口可能会对程序的可移植性造成影响，因此不建议过多的使用运行时库接口。所有的运行时接口的功能均有相应的编译指示功能对应。

5 使用及编程指南

5.1 使用 OpenACC*进行应用移植和开发的步骤

使用 OpenACC*进行应用移植通常遵循如下步骤：

- 1) 调试通过多核程序（串行或消息并行）；
- 2) 找到可并行的核心循环，通常通过以下几个方面来确定：
 - a) 核心代码必须为串行代码，由于加速设备内通常没有消息通信机制，在核心代码中不能包含有使用消息通信功能的代码；
 - b) 可以使用类似 `gprof` 等性能工具确定热点函数并确定其中的核心代码；
 - c) 支持在核心代码中的函数调用，但通常情况下被调用函数最好是纯函数，或者使用 `routine` 指示标注被调用的函数。
- 3) 在核心循环前加上 “`#pragma acc parallel loop`” 编译指示；
- 4) 确定核心代码中需要私有化的变量，并添加 `private` 子句；
 - a) 在私有化变量分析方面，用户可以使用 `-priv` 选项编译，调用 SWACC 编译系统私有化分析功能，SWACC 编译系统会进行私有化变量分析，并将显示分析结果（由于编译器分析存在局限性，需要对分析结果进行确认，以免出现遗漏或错误的情况）；
 - b) 在数组访问分析方面，用户可以额外添加 `-arrayAnalyse` 选项使用 SWACC 编译系统的分析功能进行分析。
- 5) 编译并运行，如果私有化变量添加正确，运行结果是正确的。

至此，程序移植初步完成，简单的说主要是三个步骤：1)找到核心循环；2)添加 `parallel loop` 指示，并确定 `private` 变量；3)编译运行，如果不正确检查第 2)步有无遗漏，直到程序可以正确的在加速核心上运行。

后续步骤是对程序的性能进行优化。程序在加速设备上的运行性能与下面一些因素相关：

- (1) 是否充分利用加速设备上的高速设备内存；（关键数据尽量放在 LDM）
- (2) 数据传输和计算比例；（计算访存比，通常与程序的算法和实现有关）
- (3) 合理设置循环分块大小；（灵活使用 `tile`，将关键数据尽可能多的放在 LDM，循环分块会影响放入 LDM 中的数据量）
- (4) 设法提高数据传输效率。（使用打包子句将分散的数据整合成大块的数据、使用转置子句将不连续的数据访问变成连续的数据访问）

在程序移植优化方面，需要分析核心循环内的数据访问情况，适当的添加 `copy`、`local`、`pack` 等指示，将核心数据导入设备内存中进行访问和计算，OpenACC*中程序移植过程中常见用法与编译指示用法如表 5-1 所示。

表 5-1 OpenACC*常见用法与编译指示/子句对照表

序号	功能	编译指示子句用法	说明
1	在 LDM 中为临时变量	<code>local</code> (变量列表), 用法及	在加速代码中需要被改写后再使

	(标量、小数组)申请空间	示例参见 3.17.2[P56]。	用, 并且其数值不需要更新回主存中的变量, 包括数组和标量, 可以使用 <code>local</code> 子句在 LDM 中创建相应的存储空间, <code>local</code> 中的变量没有初始值。
2	只读的标量拷贝到 LDM 中	<code>copyin</code> (变量列表), 用法及示例参见 3.17.4 [P59]。	<code>copyin</code> 的标量将在 LDM 中申请空间, 并将原始变量的值赋给 LDM 中对应的变量。
3	只读小数组整个拷贝到 LDM	<code>copyin</code> (变量列表) <code>annotate(entire(变量列表))</code> , 用法及示例参见 3.17.10.2 [P70]。	<code>copyin</code> 的数组默认会进行分块拷贝, 当有小数组需要整个拷贝入 LDM 时, 采用 <code>entire</code> 暗示的方式通知编译器。此时对应的数组将在每个加速线程的 LDM 中申请空间, 并将原始数组的数值拷贝到 LDM 中。
4	数组拷贝入拷贝出 LDM	<code>copy/copyin/copyout</code> , 用法及示例参见 3.17.4 [P59]。	按照循环的划分方式将数组分割, 以数据片段的方式在 LDM 中申请空间, 并在主存和 LDM 间进行数据传输。如果数组与被划分的循环非直接线性仿射则无法对数据进行分割。
5	最外层循环量较小, 并行度不够	<code>collapse</code> 子句, 用法及示例参见 3.5.4 [P33]。	<code>collapse</code> 的作用是将紧嵌套的若干循环合并进行并行划分, 适用于最外层循环量较小, 而且紧嵌套若干层可并行的循环的情况。
6	LDM 空间没用满	增加 <code>tile</code> 子句, 或者增大 <code>tile</code> 的分块大小, 用法及示例参见 3.5.5 [P34]。	循环的 <code>tile</code> 大小会直接影响到与之关联的数组的分块情况, 增大 <code>tile</code> 的块大小可以将每次拷贝入拷贝出 LDM 的数据增大。
7	<code>parallel copy</code> 的多维数组太大, LDM 存放不下	对数组访问相关的内层循环前添加 <code>loop</code> 指示, 并根据需要使用 <code>tile</code> 子句, 用法及示例参见 3.5 [P28]、3.5.5 [P34]。	当仅对最外层循环划分时, 某些多维数组的数据量过大, 无法一次性放入 LDM, 则可以在与数组访问相关的内层循环前添加 <code>loop</code> 指示, 用于将对应的循环分块处理, 相应的会对相关的数据也分块, 则可以将大数据划分成小的数据块。
8	数组访问与并行循环无关, 与内层循环相关, 使用 <code>parallel copy</code> 无法拷贝入 LDM	<code>loop</code> 指示、 <code>tilemask</code> 暗示子句、 <code>tile</code> 子句, 用法及示例参见 3.5 [P28]、3.17.10.1 [P68]、3.5.5 [P34]。	当数组访问与最外层并行循环无关、仅与内层循环相关时, 使用 <code>parallel copy</code> 编译器无法确定数组如何分块存储, 整个数组通常无法整个放到 LDM 中。则可以在与数组访问相关的内层循环前添加 <code>loop</code> 指示, 用于将对应的循环分块, 并根据需要设置 <code>tile</code> 子句。另

			外,在某些情况下,又希望对某些数组屏蔽新增加的 loop tile 信息,可以使用 tilemask 子句,提升某些数组的数据传输效率。
9	数组访问的数据区间是在运行时确定的	data copy , 用法及示例参见 3.17.4 [P59]。	这种情况下需要动态的数据传输,可以使用 data copy 在需要数据的代码前描述所需的数据信息,并可以根据需要使用 if 子句控制数据的重用。
10	离散访问的数组无法放入 LDM, 只读或者无写冲突的情况	group_shared, 用法及示例参见 3.17.10.7[75]。	group_shared 将会使用 LDM 共享空间存放被访问的数组, LDM 共享空间是每个加速线程划分出一块本地 LDM 空间拼接组成的, 相比本地 LDM 容量上有一定优势, 但是访问性能会有一些影响。
11	离散访问的数组无法放入单个加速线程的 LDM, 更新写(+=或-=), 可能有写冲突。	dis_reduction, 用法及示例参见 3.17.8。	对象数组的长度超过单个加速线程可用局存大小, 一般情况下需要放在主存, 频繁访问主存会导致性能下降。分布规约的原理是把对象数组分布存放在所有加速线程的局存中, 并利用核间通信机制, 保证对不在本地的数据的特定模式访问。
12	离散访问的数组无法放入 LDM, 在局部有一定的数据连续性	cache 子句, 用法及示例参见 3.4.5 [P20]。	对于离散访问的数组, 编译器无法知道拷贝哪些数据到 LDM 中, 因此 copy 子句不适用这种情况, 可以使用 cache 子句解决这个问题, 另外根据程序特点可以使用 readonly 暗示子句对 cache 进行优化, 只读的 cache 数据不用刷新会主存。
13	加速函数内需要用到 parallel copy 的数据	data present, 用法及示例参见 3.17.7 [P66]。	当 parallel copy 的数据在 parallel 内所调用的函数中需要使用时, 需要实现在 parallel 中拷贝到 LDM 中的数据在加速函数内的重用和映射, 可以用 data present 描述重用的数据信息。
14	具有多个访问模式相同的标量或数组, 想进一步提升数据传输效率	pack/packin/packout 子句, 用法及示例参见 3.4.7 [P22]。	数据打包子句的作用是将具有相同数据访问模式的变量打包成新的数据结构, 可以实现一次数据传输拷贝多个数据对象的目的。另外, 当加速段本身在循环内、且打包的数据在该循环内不改变时, 可以使用 data index 设置数据点, 将

			打包操作外提到循环之外。
15	优化跨步数组访问的性能	swap/swapin/swapout 子句，用法及示例参见 3.4.8 [P24]。	当二维数组是按列访问时，其访问属于跨步访问，高维数组以此类推。这种情况下访存及数据传输效率较低，可以使用转置子句对原始数据重新布局改善数据传输效率。另外，当加速段本身在循环内、且被转置的数据在该循环内不改变时，可以使用 data index 设置数据点，将转置操作外提到循环之外。
16	加速代码中有函数调用	routine、data copy、data present 指示，用法及示例参见 3.13 [P48]、3.6 [P35]。	加速代码中有函数调用的情况有三种处理方式：1)在被调用函数的定义处使用 routine 指示，表明该函数的定义将生成加速版本，同时可以根据情况使用 data present 和 data copy 来控制函数内的数据重用和传输；2)将函数内联；3)将函数改造成纯函数，即函数所需的数据均通过参数传递。
17	私有数组太大，无法放入 LDM	组合子句，用法及示例参见 3.17.11 [P80]。	当程序中需要私有化的临时数组很大，使用 local 无法将该数组存放到 LDM 时，可以使用组合子句，将该数组先 private，保证其私有属性，之后根据情况使用 copy 或者 cache 将其数据分段导入 LDM，保证程序性能。

5.2 程序移植示例

使用矩阵乘程序来说明使用 OpenACC*进行程序移植和优化的过程，程序代码如图 5-1 所示。

```

1 /*
2  * FILE:ex_5_2.c
3  */
4
5 #include <stdio.h>
6 #include <stdlib.h>
7 #include <sys/time.h>
8
9 #define M 1024
10 #define N 1024
11 #define K 1024
12
13 int A[M][N];
14 int B[N][K];
15 int C[M][K];
16

```

```

17 // 计时函数
18 double
19 timer()
20 {
21     double t;
22     struct timeval tv;
23     gettimeofday(&tv, NULL);
24     t = (double)tv.tv_sec*1.0+(double)tv.tv_usec*1e-6;
25     return t;
26 }
27
28 int main()
29 {
30     int i,j,k;
31     double t1, t2;
32
33     // init A
34     for(i = 0; i < M; i++)
35         for(j = 0; j < N; j++)
36             A[i][j] = i;
37
38     // init B
39     for(i = 0; i < N; i++)
40         for(j = 0; j < K; j++)
41             B[i][j] = j;
42
43     t1 = timer();
44     // matrix multiply, C=A*B
45
46     for(i = 0; i < M; i++)
47     {
48
49         for(k = 0; k < K; k++)
50         {
51
52             for(j = 0; j < N; j++)
53             {
54                 C[i][k] += A[i][j] * B[j][k];
55             }
56         }
57     }
58     t2 = timer();
59     printf("Matrix-Multiply A[%d][%d] * B[%d][%d], use time:%.2f\n",
M,N,N,K,t2-t1);
60     return 0;
61 }

```

图 5-1 矩阵乘串行程序代码

图中例子的核心段是 46~57 行的循环段，即进行矩阵乘运算的代码段。另外，为了比较移植和优化前后的程序性能，在核心代码段前后增加了计时代码，将图中的例子保存为 matrixMul.c。

1) 移植

(1) 首先使用 swacc 调试该串行程序，并记录串行版本的运行时间。

■ swacc matrixMul.c -O3;

- 提交到运算节点以单进程方式执行，运行时间：**T_Serial = 4.10 秒**（注意：运行时间与课题数据规模、运行环境、编译环境等多个因素有关，仅供参考）。
- (2) 找到核心循环，并在核心循环前添加**#pragma acc parallel loop** 指示，保存。
 - 在 `matrixMul.c` 的第 45 行添加**#pragma acc parallel loop** 指示。
- (3) 使用“`swacc -priv`”选项编译程序，根据私有变量自动分析结果或通过手工分析，添加 `private` 子句。所谓需要私有化的变量就是每个加速线程都可能会去改写，并且改写的内存区域有重叠的变量。具体到 `matrixMul.c`，需要私有化的变量有三个：`i`、`j`、`k`。
 - 在 `matrixMul.c` 第 40 行的编译指示中添加私有化子句：**#pragma acc parallel loop private(i, j, k)**
- (4) 保存，使用 `swacc` 编译，并使用 1 个主核加 64 个从核的规模运行，此时程序已经可以正确的在申威众核处理器上运行。

2) 优化

优化的基本思想是将程序中的关键数据尽可能多的放到 LDM 中，并设法提高程序中数据的传输效率。

- (1) **将标量、小数组放入 LDM:** 对于这个程序只需要将 `private` 子句换成 `local` 子句，即将 `i`、`j`、`k` 三个标量放在每个加速线程的 LDM 中，其性质仍然是加速线程私有的。
 - 第 45 行编译指示变为：**#pragma acc parallel loop local(i, j, k)**
 - 编译运行，验证正确性和效果。
- (2) **将关键数组拷贝到 LDM:** 程序中访问的数组有三个，`A`、`B`、`C`，其中 `A` 和 `B` 是只读的，`C` 是先读后写的，对于只读的数据只需要拷入(`copyin`)，只写的数据只需要拷出(`copyout`)，否则既需要拷入也需要拷出(`copy`)。在程序中第 45 行添加数据拷贝子句。

- 第 45 行编译指示变为：**#pragma acc parallel loop local(i, j, k) copyin(A, B) copy(C)**

- 编译运行，验证正确性和效果，使用同样规模运行时间约 **T_Copy = 9.02 秒**。

可以看到，程序的运行不仅没有变快，反而变慢了。注意一下编译时的屏幕输出或者使用 `-keep` 选项查看变换后的 `_slave` 中间文件，可以发现在加速代码中对 `B` 数组的访问仍然是直接访问主存数据，并没有使用 LDM 空间。

分析程序可以发现，`B` 数组的访问与并行循环 `i` 是无关的，也就是说加速线程每执行一次并行循环迭代 `i` 都需要访问 `B` 数组所有的数据，由于 LDM 容量有限，`B` 数组无法整个拷入加速线程的 LDM，因此对 `B` 数组的访问被处理成直接访问主存数据。

而 `A`、`C` 数组的访问是跟并行循环 `i` 线性相关的，也就是说跟随着并行循环 `i` 的划分，每个加速线程所需要读取的 `A` 数组的数据和需要改写的 `C` 数组的数据也是不同的，因此，对于 `A`、`C` 数组，可以仅将所需的数据放入对应加速线程的 LDM。

- (3) **解决 B 数组拷贝到 LDM 的问题:** 数组 `B` 之所以没有放入 LDM 中，是因为数组 `B` 相关的循环没有划分，导致数据太大没法放入 LDM，解决数据过大无法放入 LDM 的办法是对没有进行划分的循环使用 `tile` 进行分块，使得循环分块后对应的数据能够拷入 LDM，对于这个程序具体实现上有两种方式：

(a) 方式 1: 对 `k` 循环 `tile`，并对数组 `B` 转置拷入(`swapin`)

`B` 数组访问相关的循环变量是 `k` 和 `j`，首先可以选择对 `k` 循环 `tile`，其用意是将循环分块执行，同时可以使相关的数据按照同样的分块方式拷贝到 LDM 中。具体

做法是：

- ◆ 在第 48 行，k 循环前添加 **#pragma acc loop tile(2) annotate(tilemask(C))**
 - 最外层 i 循环的 loop 指示表明 i 循环将在多个加速线程间并行划分
 - 而内存的 k 循环上的 loop 指示表明 k 循环将在同一个加速线程内分块执行，此处 tile 指定分块大小为 2。
 - 由于数组 C 的访问也与 k 相关，对 k 循环分块之后，会影响到 C 数组的数据传输，添加 annotate(tilemask(C))的作用是，通知编译器 C 数组忽略 k 循环的分块信息，即 C 数组与 k 相关的维度（最低维）不分块，整个拷入 LDM，这样可以获得高效的数据传输。
- ◆ 修改第 45 行的编译指示，将 copyin(B)改成 swapin(B(dimension order:1, 2))
 - 第 45 行编译指示变成 **#pragma acc loop local(i,j,k) copyin(A) copy(C) swapin(B(dimension order:1, 2))**
 - 之所以使用 swapin 代替 copyin 是因为与数组 B 的访问相关的下标时 j 和 k，分别对应的是 B 数组的高维和低维，而 j 循环在 k 循环之内，也就是说 B 数组的访问是不连续的。不连续的数据访问会导致数据传输效率较低，而 swapin 的作用是对 B 数组的原始数据按照指定的维度顺序进行转置，使用转置后的数据代替原始 B 数组的访问，这样可以使不连续的数据访问变成连续的数据访问。
- ◆ 上述两个修改完成后，保存，编译运行，此时的矩阵乘时间为 **T_Tile_K = 0.45 秒**。

(b) 方式 2: 对 j 循环 tile

方式 1 是对 k 循环 tile，这个程序还可以对 B 数组的高维分块，即对应的将 j 循环分块执行。

- ◆ 在第 51 行，j 循环前添加 **#pragma acc loop tile(2) annotate(tilemask(A))**
 - 与方式 1 的含义相同，该编译指示表明将对 j 循环以 2 为块大小进行分块，相应的会影响到与 j 相关的数据拷贝，包括数组 A 和 B，同时通过 tilemask(A)的暗示子句通知编译器，A 数组忽略该分块信息，即 A 数组与 j 相关的维度不分块。
- ◆ 保存，编译运行，该版本矩阵乘的时间为 **T_Tile_J = 0.53 秒**。
- ◆ 方式 2 的运行时间之所以比方式 1 略长，是因为在方式 2 中数组 B 的访问方式仍然是先高维后低维，这对程序的性能有一定影响。
- ◆ 因此，可以尝试修改原始程序，调整 j、k 循环的顺序，之后再按照本节介绍的方法进行移植和优化。

至此，该程序基本上移植和优化完毕，优化后的核心段可以获得 9 倍左右的性能加速，后续还可以基于 SWACC 编译器生成的中间代码，进行二次开发，对核心段中的循环采用 SIMD 进行优化，二次开发的方法见后续章节，SIMD 的使用方式请参考相关用户手册。

5.3 基于 OpenACC*的二次开发

在使用 SWACC 调试通过加速程序之后，如果需要在编译器生成的中间代码的基础上进一步分析和优化改造，则可以按下列步骤操作。

1) 中间文件的建立

以 test.c 程序为例，在该文件目录下输入指令：

```
swacc test.c -dumpcommand mk
```

其中-dumpcommand 为 2.2 节介绍的 SWACC 编译选项，其作用是保留中间文件，并将对中间文件的编译命令写到 mk 文件中。

命令行执行后，将在当前目录下生成 host 程序文件和 device 程序文件，同时编译 host 和 device 程序文件的命令将被写入指定的用户自定义文件(例子中是 mk)中。host 和 device 程序文件的命名规则如下（以 c 程序为例，Fortran 类似）：

- a) host 程序文件：源文件名_host.c，如 test_host.c；
 - b) device 程序文件：源文件名_slave_行号.c，其中行号为加速区指示所在的行号，如有多个 parallel loop 指示，则会分别生成不同的 device 程序文件，如 test_slave_7.c。
- 2) 中间文件的分析改造
用户可根据需要对中间文件（host 程序文件、device 程序文件和编译命令文件）进行修改。
 - 3) 中间文件的重新编译
执行之前保留的中间文件编译过程，执行 sh 编译命令文件名（例子中为 sh mk）即可。

5.4 关于加速线程数据 Cache 的使用说明

● 硬件功能介绍

在新一代申威众核处理器中为每个从核提供了数据 Cache（DCache），可以根据需要将部分 LDM 配置成 DCache 的缓冲空间，DCache 的缓冲大小有 0KB、32KB 和 128KB 三种配置，可以根据课题需要在作业提交时或者通过 OpenACC*编译指示指定。

新一代申威众核处理器为从核 DCache 提供了有限的一致性保证，其一致性保证体现在三个方面：

- 1) 主核和从核之间：从核访问主存（gld/gst、dma、DCache 装入和淘汰）会触发主核 Cache 的一致性协议；反过来，主核对在从核 DCache 中有副本的内存访问，没有一致性保证；即从核可以拿到在主核 Cache 中的数据，但是主核拿不到从核 DCache 中的数据；
- 2) 从核和从核之间：从核间的 DCache 无关联和一致性支持，即不同从核不能修改内存同一个位置，不能有真共享；2) 提供屏蔽码机制，解决假共享的数据一致性，支持的最小访存粒度是 4B，即不同加速线程通过 DCache 以小于 4B 的粒度访问相邻但不相同的内存区域时，无法保证数据一致性。
- 3) 单从核内：从核 Cache 不保证与软件管理的 LDM 空间的一致性。从核 DMA 过程中，无法获得从核 DCache 中的最新数据。

● 基于 OpenACC*使用从核 DCache 的方式

OpenACC*中提供了对新一代申威众核处理器从核 DCache 的支持和相应的语言功能，默认情况下，对没有使用 OpenACC*拷贝子句描述的变量访问，均会通过 DCache 进行优化。对于 OpenACC*应用推荐的 DCache 使用方式和步骤如下：

- 1) 在计算核心段前正确添加 parallel loop 编译指示，并根据情况添加私有化、规约等子句，确保程序可以正确的并行加速；在运行时打开 DCache 并尽量将容量配置为 128KB，一般情况下，经过 DCache 的优化，程序可以获得不错的提升；如果加速效果比较满意，则可以跳过后续步骤。
- 2) 使用 OpenACC*提供的数据拷贝子句，将核心数据拷贝到 LDM 中访问，此类变量不会再进入 DCache；此时可根据需要减少 DCache 的容量，将其改为 32KB 或 0KB，建议

在空间允许的情况下尽量保留一定的 DCache 容量。

在使用 DCache 时，OpenACC*默认会在每个加速区退出时刷新整个从核 Dcache，OpenACC*提供的关于从核 DCache 相关的功能如下表所示。

表 5-2 OpenACC*中从核 DCache 相关编译指示/子句一览

序号	功能	编译指示子句用法	说明
1	将 DCache 中的数据副本更新回主存	flush 指示，用法及示例参见 3.16 [P53]。	flush 指示在加速计算区代码中使用，执行后 Cache 中所有的数据将更新回主存并将 Cache 状态置为无效。在每个加速计算区结束时默认刷新加速线程的数据 Cache。
2	指定变量不通过 DCache 访问	uncache (变量列表)，用法参见 3.4.15 [P28]、3.11.1 [P47]、3.13.3 [P52]。	该子句可以作为 parallel、routine、atomic 的子句出现，且只在对应指示的作用范围内有效。将指定的变量描述为不可 Cache 属性，通知编译器尽量不适用 Cache 优化加速计算区内的访存操作。在某些情况下，使用 Cache 可能会带来性能的损失或者正确性的问题，可以尝试使用。
3	调整 DCache 的容量	spm_config，用法参见 3.17.10.8 [P76]。	运行时动态调整 DCache 的容量，以适应不同加速区对 DCache 不同的需求。

● 使用从核 DCache 的注意事项，重要!!!

1) 程序中不能通过变量名之外的方式访问不可 Cache 变量，比如指针、别名等方式，否则可能会导致一致性问题；在加速段中，若将全局变量标记为不可 Cache 属性，除在 parallel 上使用 uncache 子句进行描述外，还需在该加速段调用的函数内通过 routine 上的 uncache 子句进行描述，以保证变量的单一属性；

2) 重点关注对小于 4B 数据类型变量的交错写访问；当程序中有小于 4B 数据类型的数组，并且该数组在加速区内由加速线程共享访问、访问位置无重叠时，代码本身并没有问题，但是由于硬件 DCache 无法保证小于 4B 粒度假共享的数据一致性，运行时有可能有正确性问题；

3) 在加速区内同一个变量的访问模式只能有一种，要么使用 copy 等指示进行 DMA 处理，要么直接访存通过 DCache 优化；

4) 动态调整 LDM 配置时，通过 enter data 等方式驻留在 LDM 中的数据有可能丢失，因此，不建议在非结构数据区作用范围内调整 LDM 配置。另外，动态调整 LDM 配置存在一定的开销，建议谨慎使用。